materials-science-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `materials_science_ai999x.about()` for the same answer.

Every capability materials-science-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import materials_science_ai999x

result = materials_science_ai999x.solve('cubic_lattice_parameter', structure='FCC', R='1.278e-10 m')
result.show()
# a = 3.61473e-10 m, a_nm = 0.361473, nearest_neighbour_nm = 0.2556
```

By topic

topic	capabilities
Ceramics / crystal structures	Lattice parameter and density of an ionic crystal
Ceramics / ionic structures	Coordination number from the cation-anion radius ratio
Composites / elastic moduli	Elastic moduli of a continuous-fibre composite (rule of mixtures)
Composites / fibre reinforcement	Critical fibre length
Crystal structures / crystal geometry	Angle between two directions in a cubic crystal, Angle between two lattice directions in any crystal, Angle between two lattice planes in any crystal
Crystal structures / cubic lattices	Lattice parameter from the atomic radius
Crystal structures / density	Theoretical density of a metal, Lattice parameter and atomic radius from the density
Crystal structures / hexagonal cells	Volume and density of a hexagonal close-packed cell
Crystal structures / hexagonal indices	Three-index to four-index (Miller-Bravais) direction, Four-index (Miller-Bravais) to three-index direction
Crystal structures / linear and planar density	Linear density along a crystal direction, Planar density of a crystal plane
Crystal structures / Miller indices	Miller indices of a plane from its intercepts
Crystal structures / packing	Atomic packing factor and coordination number
Crystal structures / polymorphism	Volume change of a BCC to FCC transformation
Crystal structures / symmetry	Space group of a common crystal structure

topic	capabilities
Crystal structures / unit cells	Volume of a general (triclinic) unit cell
Defects / dislocations	Burgers vector of a perfect dislocation
Defects / interstitials	Largest atom that fits an interstitial site
Defects / point defects	Equilibrium vacancy concentration, Vacancy formation energy from vacancy fractions
Diffraction / Bragg's law	Bragg angle of a reflection, X-ray wavelength from a Bragg reflection, Interplanar spacing from a Bragg reflection
Diffraction / interplanar spacing	Interplanar spacing in a cubic crystal, Interplanar spacing in a tetragonal crystal, Interplanar spacing in an orthorhombic crystal, Interplanar spacing in a hexagonal crystal, Interplanar spacing in a general (triclinic) lattice
Diffraction / structure factor	Structure factor and extinction of a cubic reflection, First allowed reflections of a cubic crystal
Diffraction / X-ray diffraction	Diffraction angle of a cubic (h k l) reflection
Diffusion / non-steady state	Concentration in a semi-infinite solid (Fick's second law), Time to reach a concentration at a depth (Fick's second law), Depth at which a concentration is reached (Fick's second law)
Diffusion / steady state	Steady-state diffusion flux (Fick's first law)
Diffusion / temperature dependence	Activation energy and pre-exponential factor of diffusion, Diffusion coefficient at a temperature (Arrhenius)
Electronic properties / metals	Fermi energy of a free-electron metal
Electronic properties / semiconductor devices	Built-in potential of a p-n junction
Electronic properties / semiconductors	Intrinsic carrier concentration of a semiconductor, Band gap from intrinsic conductivities at several temperatures, Electron and hole concentrations of a doped semiconductor
Fracture / fracture mechanics	Griffith critical stress of a brittle solid, Fracture stress from the fracture toughness, Critical crack length from the fracture toughness
Fracture / statistics of brittle fracture	Weibull probability of failure of a brittle part, Design stress for a target failure probability (Weibull), Weibull modulus from a set of measured strengths
Mechanical properties / creep	Stress exponent and activation energy of steady-state creep, Larson-Miller parameter (creep rupture)
Mechanical properties / elasticity	Elastic elongation and diameter change of a rod in tension, Shear modulus, bulk modulus and Lame constant from E and Poisson's ratio
Mechanical properties / plasticity	Strain-hardening exponent and strength coefficient (Hollomon)
Mechanical properties / slip in single crystals	Resolved shear stress and Schmid factor of a slip system, Most favourable slip system of an FCC or BCC crystal
Mechanical properties / strengthening mechanisms	Hall-Petch grain-size strengthening
Mechanical properties / tensile test	True stress and true strain from engineering values, Young's modulus from elastic stress-strain data, 0.2 % offset yield strength from a stress-strain curve, Toughness (area under the stress-strain curve), Modulus of resilience
Mechanical properties / thermal stresses	Thermal stress in a constrained body
Phase diagrams / composition	Weight percent to atomic percent (binary alloy), Atomic percent to weight percent (binary alloy)
Phase diagrams / eutectic systems	Primary and eutectic microconstituent fractions

topic	capabilities
Phase diagrams / iron-carbon system	Proeutectoid phase and pearlite in a plain-carbon steel
Phase diagrams / phase rule	Degrees of freedom by Gibbs' phase rule
Phase diagrams / two-phase regions	Phase fractions by the lever rule
Phase transformations / kinetics	Avrami (JMAK) transformation kinetics
Phase transformations / nucleation	Critical nucleus radius and activation energy (homogeneous nucleation)
Polymers / chain conformation	End-to-end distance of a freely rotating chain
Polymers / copolymers	Average repeat-unit weight and degree of polymerization of a copolymer
Polymers / crystallinity	Percent crystallinity of a polymer from densities
Polymers / elastomers	Stress in a stretched rubber (affine network)
Polymers / molecular weight	Number- and weight-average molecular weight and dispersity, Repeat-unit molecular weight and degree of polymerization
Polymers / viscoelasticity	Stress relaxation of a Maxwell viscoelastic solid, Creep strain of a Kelvin-Voigt viscoelastic solid
Thermal properties / heat capacity	Molar heat capacity of a solid (Debye model), Molar heat capacity of a solid (Einstein model)
Thermal properties / thermal conductivity	Electronic thermal conductivity of a metal (Wiedemann-Franz law)
Thermal properties / thermal expansion	Linear thermal expansion, Volumetric thermal expansion

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
cubic_lattice_parameter	Lattice parameter from the atomic radius	structure, R	a, a_nm, nearest_neighbour_nm
atomic_packing_factor	Atomic packing factor and coordination number	structure	apf, coordination, atoms_per_cell
theoretical_density	Theoretical density of a metal	structure, A, R	rho, rho_gcc, a_nm
lattice_parameter_from_density	Lattice parameter and atomic radius from the density	structure, rho, A	a, a_nm, R_nm
hcp_cell_density	Volume and density of a hexagonal close-packed cell	a, c, A	V, V_nm3, rho, rho_gcc, c_over_a
unit_cell_volume	Volume of a general (triclinic) unit cell	a, b, c, alpha, beta, gamma	V, V_nm3
d_spacing_cubic	Interplanar spacing in a cubic crystal	a, h, k, l	d, d_nm
d_spacing_tetragonal	Interplanar spacing in a tetragonal crystal	a, c, h, k, l	d, d_nm

id	operation	inputs	outputs
d_spacing_orthorhombic	Interplanar spacing in an orthorhombic crystal	a, b, c, h, k, l	d, d_nm
d_spacing_hexagonal	Interplanar spacing in a hexagonal crystal	a, c, h, k, l, i	d, d_nm
d_spacing_triclinic	Interplanar spacing in a general (triclinic) lattice	a, b, c, alpha, beta, gamma, h, k, l	d, d_nm
bragg_angle	Bragg angle of a reflection	d, lam, n	theta_deg, two_theta_deg
bragg_wavelength	X-ray wavelength from a Bragg reflection	d, theta, n	lam, lam_nm
bragg_spacing	Interplanar spacing from a Bragg reflection	lam, theta, n	d, d_nm
cubic_reflection_angle	Diffraction angle of a cubic (h k l) reflection	a, h, k, l, lam	d_nm, theta_deg, two_theta_deg
structure_factor_cubic	Structure factor and extinction of a cubic reflection	structure, h, k, l	F_over_f, F2_over_f2, allowed
allowed_reflections_cubic	First allowed reflections of a cubic crystal	structure, count	N_values, N_last, N_first
linear_density	Linear density along a crystal direction	structure, u, v, w, a	LD_per_nm, LD, repeat_nm, LPF
planar_density	Planar density of a crystal plane	structure, h, k, l, a	PD_per_nm2, PD, PPF
miller_indices_from_intercepts	Miller indices of a plane from its intercepts	x, y, z	h, k, l, indices
miller_bravais_direction	Three-index to four-index (Miller-Bravais) direction	U, V, W	u, v, t, w, indices
miller_bravais_to_three_index	Four-index (Miller-Bravais) to three-index direction	u, v, t, w	U, V, W, indices
angle_between_directions_cubic	Angle between two directions in a cubic crystal	u1, v1, w1, u2, v2, w2	angle_deg
angle_between_directions	Angle between two lattice directions in any crystal	a, b, c, alpha, beta, gamma, u1, v1, w1, u2, v2, w2	angle_deg
angle_between_planes	Angle between two lattice planes in any crystal	a, b, c, alpha, beta, gamma, h1, k1, l1, h2, k2, l2	angle_deg
interstitial_site_size	Largest atom that fits an interstitial site	structure, site, R	r, r_nm, ratio
radius_ratio_coordination	Coordination number from the cation-anion radius ratio	rc, ra	ratio, CN, geometry
ceramic_density	Lattice parameter and density of an ionic crystal	prototype, rc, ra, Ac, Aa	a, a_nm, rho, rho_gcc
polymorphic_volume_change	Volume change of a BCC to FCC transformation	R_bcc, R_fcc	volume_change_percent, V_atom_bcc_nm3, V_atom_fcc_nm3

id	operation	inputs	outputs
burgers_vector	Burgers vector of a perfect dislocation	structure, a	b, b_nm, direction
space_group	Space group of a common crystal structure	structure	number, symbol, crystal_system
vacancy_concentration	Equilibrium vacancy concentration	rho, A, Qv, T	N, Nv, fraction, log10_Nv, N_per_cm3
vacancy_formation_energy_fit	Vacancy formation energy from vacancy fractions	T, fraction	Qv_eV, Qv, prefactor, r_squared
fick_first_law_flux	Steady-state diffusion flux (Fick's first law)	D, C1, C2, x1, x2	J, J_per_hour
fick_second_law_concentration	Concentration in a semi-infinite solid (Fick's second law)	C0, Cs, x, D, t	Cx, z, ratio
diffusion_time_to_concentration	Time to reach a concentration at a depth (Fick's second law)	C0, Cs, Cx, x, D	t, t_hours, z
diffusion_depth_of_concentration	Depth at which a concentration is reached (Fick's second law)	C0, Cs, Cx, D, t	x, x_mm, z
diffusion_arrhenius_fit	Activation energy and pre-exponential factor of diffusion	T, D, T_target	Q, Q_kJmol, D0, log10_D0, r_squared, D_target, log10_D_target
true_stress_and_strain	True stress and true strain from engineering values	stress, strain	true_stress, true_stress_MPa, true_strain
elastic_deformation_of_a_rod	Elastic elongation and diameter change of a rod in tension	F, d0, l0, E, nu	stress_MPa, axial_strain, elongation_mm, lateral_strain, diameter_change_um
isotropic_elastic_constants	Shear modulus, bulk modulus and Lame constant from E and Poisson's ratio	E, nu	G, G_GPa, K, K_GPa, lame_GPa
youngs_modulus_from_data	Young's modulus from elastic stress-strain data	strain, stress	E_GPa, E, intercept_MPa, r_squared
offset_yield_strength	0.2 % offset yield strength from a stress-strain curve	strain, stress, E, offset	yield_MPa, yield_strain
tensile_toughness	Toughness (area under the stress-strain curve)	strain, stress	toughness_MJ_m3, toughness, uts_MPa, fracture_strain
modulus_of_resilience	Modulus of resilience	sy, E	Ur, Ur_MJ_m3, yield_strain
strain_hardening_exponent	Strain-hardening exponent and strength coefficient (Hollomon)	true_strain, true_stress, strain_target	n, K_MPa, r_squared, stress_target_MPa
hall_petch	Hall-Petch grain-size strengthening	d_um, yield_MPa, d_target_um	sigma0_MPa, k_MPa_sqrt_um, k_MPa_sqrt_m, r_squared, yield_target_MPa
thermal_stress	Thermal stress in a constrained body	E, alpha, dT, constraint, nu	stress, stress_MPa
composite_rule_of_mixtures	Elastic moduli of a continuous-fibre	Ef, Em, Vf	E_long_GPa, E_trans_GPa, fiber_load_fraction, E_long, E_trans

id	operation	inputs	outputs
	composite (rule of mixtures)		
critical_fiber_length	Critical fibre length	sigma_f, d, tau_c	lc, lc_mm, aspect_ratio
griffith_critical_stress	Griffith critical stress of a brittle solid	E, gamma_s, a	sigma_c, sigma_c_MPa
fracture_critical_stress	Fracture stress from the fracture toughness	K_IC, a, Y	sigma_c, sigma_c_MPa
critical_crack_length	Critical crack length from the fracture toughness	K_IC, sigma, Y	a_c, a_c_mm
weibull_failure_probability	Weibull probability of failure of a brittle part	sigma, sigma0, m, V_ratio	P_failure, P_survival
weibull_design_stress	Design stress for a target failure probability (Weibull)	P_failure, sigma0, m, V_ratio	sigma, sigma_MPa
weibull_modulus_fit	Weibull modulus from a set of measured strengths	strengths	m, sigma0, r_squared
creep_parameters_fit	Stress exponent and activation energy of steady-state creep	stress_MPa, T, rate	n, Q_kJmol
maxwell_stress_relaxation	Stress relaxation of a Maxwell viscoelastic solid	sigma0, E, eta, t	sigma_t, sigma_t_MPa, tau, ratio
kelvin_voigt_creep	Creep strain of a Kelvin-Voigt viscoelastic solid	sigma0, E, eta, t	strain, fraction_of_elastic, tau
schmid_factor	Resolved shear stress and Schmid factor of a slip system	load, plane, direction, sigma, tau_crss	schmid_factor, phi_deg, lambda_deg, tau_R_MPa, yield_MPa
max_schmid_factor	Most favourable slip system of an FCC or BCC crystal	structure, load, tau_crss	max_schmid_factor, active_systems, systems, yield_MPa
lever_rule	Phase fractions by the lever rule	C0, C1, C2, mass	W1, W2, m1, m2
eutectic_microconstituents	Primary and eutectic microconstituent fractions	C0, C_alpha, C_E, C_beta	W_primary, W_eutectic, W_alpha_total, W_beta_total, primary_phase
iron_carbon_microconstituents	Proeutectoid phase and pearlite in a plain-carbon steel	C0	W_proeutectoid, W_pearlite, W_ferrite_total, W_cementite_total, proeutectoid_phase
gibbs_phase_rule	Degrees of freedom by Gibbs' phase rule	C, P, pressure	F
homogeneous_nucleation	Critical nucleus radius and activation energy (homogeneous nucleation)	gamma, dHf, Tm, T	undercooling, dGv_MJ_m3, r_star, r_star_nm, dG_star, dG_star_eV
avrami_kinetics	Avrami (JMAK) transformation kinetics	t, y, y_target	n, k, log10_k, t_half, r_squared, t_target
weight_to_atomic_percent	Weight percent to atomic percent (binary alloy)	element1, element2, wt1	at1, at2
atomic_to_weight_percent	Atomic percent to weight percent (binary alloy)	element1, element2, at1	wt1, wt2

id	operation	inputs	outputs
molecular_weight_averages	Number- and weight-average molecular weight and dispersity	M, fraction, basis	Mn, Mw, PDI
degree_of_polymerization	Repeat-unit molecular weight and degree of polymerization	formula, Mn	repeat_unit_g_mol, DP
copolymer_degree_of_polymerization	Average repeat-unit weight and degree of polymerization of a copolymer	formula1, formula2, f1, Mn	m_avg_g_mol, DP
polymer_crystallinity	Percent crystallinity of a polymer from densities	rho_s, rho_c, rho_a	crystallinity_percent
rubber_elasticity	Stress in a stretched rubber (affine network)	rho, Mc, T, stretch	G_MPa, stress_MPa, true_stress_MPa
polymer_chain_dimensions	End-to-end distance of a freely rotating chain	n, l, theta	r_rms, r_rms_nm, C_n, contour_nm
intrinsic_carrier_concentration	Intrinsic carrier concentration of a semiconductor	Eg, T, me, mh	Nc, Nv, ni, log10_ni, log10_Nc, Ei_offset_meV
band_gap_from_conductivity	Band gap from intrinsic conductivities at several temperatures	T, sigma	Eg_eV, prefactor, r_squared
wiedemann_franz	Electronic thermal conductivity of a metal (Wiedemann-Franz law)	T, sigma, rho	kappa, lorenz
debye_heat_capacity	Molar heat capacity of a solid (Debye model)	theta_D, T	Cv, Cv_over_3R
einstein_heat_capacity	Molar heat capacity of a solid (Einstein model)	theta_E, T	Cv, Cv_over_3R
fermi_energy_free_electron	Fermi energy of a free-electron metal	rho, A, valence	n, log10_n, EF, EF_eV, TF, vF
arrhenius_diffusivity	Diffusion coefficient at a temperature (Arrhenius)	D0, Q, T	D, log10_D, Q_over_RT
larson_miller_parameter	Larson-Miller parameter (creep rupture)	T, t_r, C, T2	LMP, LMP_thousands, log10_t_hours, t_r2_hours, log10_t2_hours
linear_thermal_expansion	Linear thermal expansion	alpha, L0, dT	dL, dL_mm, L, strain
volumetric_thermal_expansion	Volumetric thermal expansion	alpha, V0, dT	beta, dV, dV_cm3, V, dV_exact, dV_exact_cm3
extrinsic_carrier_concentration	Electron and hole concentrations of a doped semiconductor	Nd, Na, ni, T	n, p, log10_n, log10_p, carrier_type, EF_minus_Ei_eV
pn_junction_built_in_potential	Built-in potential of a p-n junction	Na, Nd, ni, T, eps_r	V_bi, V_bi_approx, thermal_voltage, W, W_um, x_n_um, x_p_um

Lattice parameter from the atomic radius — `cubic_lattice_parameter`

The cubic lattice parameter a of an SC, BCC, FCC or diamond-cubic crystal of hard-sphere atoms of radius R.

$$a_{\mathrm{SC}} = 2R,\; a_{\mathrm{BCC}} = \frac{4R}{\sqrt{3}},\; a_{\mathrm{FCC}} = 2\sqrt{2}\,R,\; a_{\mathrm{dia}} = \frac{8R}{\sqrt{3}}$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC, diamond)	
`R`	atomic radius	

output	meaning
`a`	lattice parameter
`a_nm`	lattice parameter in nm
`nearest_neighbour_nm`	nearest-neighbour distance 2R in nm

- **Method:** atoms touch along the close-packed direction: SC $a = 2R$, BCC $a = 4R/sqrt(3)$, FCC $a = 2$ sqrt(2) R, diamond $a = 8R/sqrt(3)$
- **Assumes:** $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('cubic_lattice_parameter', structure='FCC', R=1.278e-10)
# -> a = 3.61473e-10 m, a_nm = 0.361473, nearest_neighbour_nm = 0.2556   (SOLVED)
```

```
materials_science_ai999x.solve('cubic_lattice_parameter', structure='BCC', R=1.241e-10)
# -> a = 2.86597e-10 m, a_nm = 0.286597, nearest_neighbour_nm = 0.2482   (SOLVED)
```

```
materials_science_ai999x.solve('cubic_lattice_parameter', structure='SC', R=1.67e-10)
# -> a = 3.34e-10 m, a_nm = 0.334, nearest_neighbour_nm = 0.334   (SOLVED)
```

```
materials_science_ai999x.solve('cubic_lattice_parameter', structure='diamond', R=1.176e-10)
# -> a = 5.43171e-10 m, a_nm = 0.543171, nearest_neighbour_nm = 0.2352   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lattice parameter from atomic radius: structure = FCC, R = 0.1278 nm	SOLVED	a = 3.61473e-10 m, a_nm = 0.361473, nearest_neighbour_nm = 0.2556
Lattice parameter from atomic radius: structure = BCC, R = 1.241 angstrom	SOLVED	a = 2.86597e-10 m, a_nm = 0.286597, nearest_neighbour_nm = 0.2482
Lattice parameter from atomic radius: structure = FCC, R = -1 nm	INVALID_INPUT	Lattice parameter from the atomic radius: the atomic radius must be a positive number.
Lattice parameter from atomic radius: structure = FCC	NEEDS_INPUT	cubic_lattice_parameter needs R (atomic radius). Example: solve('cubic_lattice_parameter', structure='FCC', R='1.278e-10

question	outcome	result
Lattice parameter from atomic radius: structure = FCC, R = 0.1278	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: materials_science_ai999x.template

Atomic packing factor and coordination number — `atomic_packing_factor`

The fraction of a crystal's volume filled by hard-sphere atoms, the coordination number and the atoms per conventional cell of SC, BCC, FCC, HCP or diamond cubic.

$$\mathrm{APF} = \frac{n\,\frac{4}{3}\pi R^3}{V_{\mathrm{cell}}}$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC, HCP, diamond)	

output	meaning
`apf`	atomic packing factor
`coordination`	coordination number
`atoms_per_cell`	atoms per conventional cell (HCP: the hexagonal prism)

- **Method:** APF = n $(4/3)$ pi R^3 / V_cell with the cell edge written in R
- **Assumes:** hard spheres touching along the close-packed direction; ideal c/a for HCP
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('atomic_packing_factor', structure='FCC')
# -> apf = 0.74048, coordination = 12, atoms_per_cell = 4   (SOLVED)
```

```
materials_science_ai999x.solve('atomic_packing_factor', structure='BCC')
# -> apf = 0.680175, coordination = 8, atoms_per_cell = 2   (SOLVED)
```

```
materials_science_ai999x.solve('atomic_packing_factor', structure='SC')
# -> apf = 0.523599, coordination = 6, atoms_per_cell = 1   (SOLVED)
```

```
materials_science_ai999x.solve('atomic_packing_factor', structure='HCP')
# -> apf = 0.74048, coordination = 12, atoms_per_cell = 6   (SOLVED)
```

```
materials_science_ai999x.solve('atomic_packing_factor', structure='diamond')
# -> apf = 0.340087, coordination = 4, atoms_per_cell = 8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Atomic packing factor: structure = FCC	SOLVED	apf = 0.74048, coordination = 12, atoms_per_cell = 4

question	outcome	result
Atomic packing factor: structure = diamond	SOLVED	apf = 0.340087, coordination = 4, atoms_per_cell = 8
Atomic packing factor: structure = BCT	INVALID_INPUT	structure: one of SC, BCC, FCC, HCP, diamond; got 'bct'.
Atomic packing factor:	NEEDS_INPUT	atomic_packing_factor needs structure (crystal structure (SC, BCC, FCC, HCP, diamond)). Example: solve('atomic_packing_f

Theoretical density of a metal — `theoretical_density`

The density of a crystal from its structure, the atomic radius and the molar mass: $\rho = n$ A / (V_cell N_A).

$$\rho = \frac{nA}{V_{\text{cell}} N_A}$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC, HCP, diamond)	
`A`	molar mass (atomic weight)	
`R`	atomic radius	

output	meaning
`rho`	theoretical density
`rho_gcc`	density in g/cm^3
`a_nm`	lattice parameter a in nm

- **Method:** a from R (hard spheres), V_cell = a^3 (HCP: (3 sqrt(3)/2) a^2 c with $c = sqrt(8/3)$ a), $\rho = n$ A/(V_cell N_A)
- **Assumes:** $A > 0$; $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('theoretical_density', structure='FCC', A=0.06355, R=1.28e-10)
# -> rho = 8895.28 kg/m^3, rho_gcc = 8.89528, a_nm = 0.362039   (SOLVED)
```

```
materials_science_ai999x.solve('theoretical_density', structure='BCC', A=0.05585, R=1.241e-10)
# -> rho = 7879.32 kg/m^3, rho_gcc = 7.87932, a_nm = 0.286597   (SOLVED)
```

```
materials_science_ai999x.solve('theoretical_density', structure='HCP', A=0.02431, R=1.6e-10)
# -> rho = 1742.2 kg/m^3, rho_gcc = 1.7422, a_nm = 0.32   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Theoretical density: structure = FCC, A = 63.55 g/mol, R = 0.128 nm	SOLVED	rho = 8895.28 kg/m^3, rho_gcc = 8.89528, a_nm = 0.362039
Theoretical density: structure = BCC, A = 55.85 g/mol, R = 1.241 angstrom	SOLVED	rho = 7879.32 kg/m^3, rho_gcc = 7.87932, a_nm = 0.286597

question	outcome	result
Theoretical density: structure = FCC, A = 63.55 g/mol	NEEDS_INPUT	theoretical_density needs R (atomic radius). Example: solve('theoretical_density', structure='FCC', A='0.06355 kg/mol',
Theoretical density: structure = FCC, A = 0 g/mol, R = 0.128 nm	INVALID_INPUT	Theoretical density of a metal: the molar mass must be a positive number.

Lattice parameter and atomic radius from the density — `lattice_parameter_from_density`

The lattice parameter a and hard-sphere atomic radius R of a crystal of known structure, density and molar mass.

$$a = \left(\frac{nA}{\rho N_A}\right)^{1/3}$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC, HCP, diamond)	
`rho`	measured density	
`A`	molar mass	

output	meaning
`a`	lattice parameter a
`a_nm`	lattice parameter a in nm
`R_nm`	atomic radius in nm

- **Method:** V_cell = n A / (rho N_A); a = V_cell^(1/3) (HCP with ideal c/a); R from the touching condition
- **Assumes:** $\rho > 0$; $A > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('lattice_parameter_from_density', structure='FCC', rho=8960.0,
A=0.06355)
# -> a = 3.61165e-10 m, a_nm = 0.361165, R_nm = 0.127691   (SOLVED)
```

```
materials_science_ai999x.solve('lattice_parameter_from_density', structure='BCC', rho=7870.0,
A=0.05585)
# -> a = 2.8671e-10 m, a_nm = 0.28671, R_nm = 0.124149   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lattice parameter from density: structure = FCC, rho = 8.96 g/cm^3, A = 63.55 g/mol	SOLVED	a = 3.61165e-10 m, a_nm = 0.361165, R_nm = 0.127691
Lattice parameter from density: structure = FCC, rho = 8.96 g/cm^3	NEEDS_INPUT	lattice_parameter_from_density needs A (molar mass). Example: solve('lattice_parameter_from_density', structure='FCC', r

Volume and density of a hexagonal close-packed cell — `hcp_cell_density`

The volume of the HCP hexagonal prism (6 atoms), the density and the c/a ratio from a, c and the molar mass.

$$V = \frac{3\sqrt{3}}{2} a^2 c, \quad \rho = \frac{6A}{V N_A}$$

input	meaning	condition
`a`	lattice parameter a	
`c`	lattice parameter c	
`A`	molar mass	

output	meaning
`V`	volume of the hexagonal prism (6 atoms)
`V_nm3`	prism volume in nm^3
`rho`	density
`rho_gcc`	density in g/cm^3
`c_over_a`	c/a ratio

- **Method:** V = (3 sqrt(3)/2) a^2 c, $\rho = 6$ A/(V N_A)
- **Assumes:** $a > 0$; $c > 0$; $A > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('hcp_cell_density', a=3.209e-10, c=5.211e-10, A=0.02431)
# -> V = 1.39416e-28 m^3, V_nm3 = 0.139416, rho = 1737.29 kg/m^3, rho_gcc = 1.73729, c_over_a =
1.62387   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
HCP cell: a = 0.3209 nm, c = 0.5211 nm, A = 24.31 g/mol	SOLVED	V = 1.39416e-28 m^3, V_nm3 = 0.139416, rho = 1737.29 kg/m^3, rho_gcc = 1.73729, c_over_a = 1.62387
HCP cell: a = 0.3209 nm, A = 24.31 g/mol	NEEDS_INPUT	hcp_cell_density needs c (lattice parameter c). Example: solve('hcp_cell_density', a='3.209e-10 m', c='5.211e-10 m', A='

Volume of a general (triclinic) unit cell — `unit_cell_volume`

The volume of a unit cell from its six lattice parameters a, b, c, alpha, beta, gamma.

$$V = abc\sqrt{1 - \cos^2\alpha - \cos^2\beta - \cos^2\gamma + 2\cos\alpha\cos\beta\cos\gamma}$$

input	meaning	condition
`a`	cell edge a	
`b`	cell edge b	

input	meaning	condition
`c`	cell edge c	
`alpha`	cell angle alpha in degrees (between b and c)	
`beta`	cell angle beta in degrees (between a and c)	
`gamma`	cell angle gamma in degrees (between a and b)	

output	meaning
`V`	cell volume
`V_nm3`	cell volume in nm^3

- **Method:** V = abc sqrt($1 - cos^2\ \alpha - cos^2\ \beta - cos^2\ \gamma + 2$ cos alpha cos beta cos gamma)
- **Assumes:** 0 < angles < 180; $1 - cos^2$(alpha) - cos^2(beta) - cos^2(gamma) + 2 cos(alpha) cos(beta) cos(gamma) > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('unit_cell_volume', a=5e-10, b=6e-10, c=7e-10, alpha=90, beta=100,
gamma=90)
# -> V = 2.0681e-28 m^3, V_nm3 = 0.20681   (SOLVED)
```

```
materials_science_ai999x.solve('unit_cell_volume', a=4e-10, b=4e-10, c=4e-10, alpha=90, beta=90,
gamma=90)
# -> V = 6.4e-29 m^3, V_nm3 = 0.064   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Unit cell volume: a = 0.5 nm, b = 0.6 nm, c = 0.7 nm, alpha = 90, beta = 100, gamma = 90	SOLVED	V = 2.0681e-28 m^3, V_nm3 = 0.20681
Unit cell volume: a = 0.5 nm, b = 0.6 nm, c = 0.7 nm, alpha = 150, beta = 150, gamma = 150	INVALID_INPUT	Volume of a general (triclinic) unit cell: the cell angles are incompatible (no cell has them).
Unit cell volume: a = 0.5 nm, b = 0.6 nm, c = 0.7 nm	NEEDS_INPUT	unit_cell_volume needs alpha, beta, gamma (cell angle alpha in degrees (between b and c); cell angle beta in degrees (be

Interplanar spacing in a cubic crystal — `d_spacing_cubic`

The spacing of the (h k l) planes of a cubic lattice: $d = a/sqrt$(h^2 + k^2 + l^2).

$$d_{hkl} = \frac{a}{\sqrt{h^2 + k^2 + l^2}}$$

input	meaning	condition
`a`	lattice parameter	
`h`	Miller index h	
`k`	Miller index k	
`l`	Miller index l	

output	meaning
`d`	interplanar spacing
`d_nm`	interplanar spacing in nm

- **Method:** $d = \frac{a}{\sqrt{h^2+k^2+l^2}}$
- **Assumes:** $a > 0$; integer indices, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('d_spacing_cubic', a=3.615e-10, h=1, k=1, l=1)
# -> d = 2.08712e-10 m, d_nm = 0.208712   (SOLVED)
```

```
materials_science_ai999x.solve('d_spacing_cubic', a=2.866e-10, h=2, k=1, l=1)
# -> d = 1.17004e-10 m, d_nm = 0.117004   (SOLVED)
```

```
materials_science_ai999x.solve('d_spacing_cubic', a=4e-10, h=2, k=0, l=0)
# -> d = 2e-10 m, d_nm = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cubic interplanar spacing: a = 0.3615 nm, h = 1, k = 1, l = 1	SOLVED	d = 2.08712e-10 m, d_nm = 0.208712
Cubic interplanar spacing: a = 2.866 angstrom, h = 2, k = 1, l = 1	SOLVED	d = 1.17004e-10 m, d_nm = 0.117004
Cubic interplanar spacing: a = 0.3615 nm, h = 0, k = 0, l = 0	INVALID_INPUT	Interplanar spacing in a cubic crystal: the Miller indices must not all be zero.
Cubic interplanar spacing: a = 0.3615 nm, h = 1, k = 1	NEEDS_INPUT	d_spacing_cubic needs l (Miller index l). Example: solve('d_spacing_cubic', a='3.615e-10 m', h=1, k=1, l=1).

Interplanar spacing in a tetragonal crystal — `d_spacing_tetragonal`

The spacing of the (h k l) planes of a tetragonal lattice: $1/d^2 = (h^2 + k^2)/a^2 + l^2/c^2$.

$$\frac{1}{d^2} = \frac{h^2 + k^2}{a^2} + \frac{l^2}{c^2}$$

input	meaning	condition
`a`	lattice parameter a	
`c`	lattice parameter c	
`h`	Miller index h	
`k`	Miller index k	
`l`	Miller index l	

output	meaning
`d`	interplanar spacing

output	meaning
`d_nm`	interplanar spacing in nm

- **Method:** $1/d^2 = (h^2 + k^2)/a^2 + l^2/c^2$
- **Assumes:** a, $c > 0$; integer indices, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('d_spacing_tetragonal', a=3e-10, c=5e-10, h=1, k=0, l=1)
# -> d = 2.57248e-10 m, d_nm = 0.257248   (SOLVED)
```

```
materials_science_ai999x.solve('d_spacing_tetragonal', a=4e-10, c=6e-10, h=0, k=0, l=2)
# -> d = 3e-10 m, d_nm = 0.3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Tetragonal interplanar spacing: a = 0.3 nm, c = 0.5 nm, h = 1, k = 0, l = 1	SOLVED	d = 2.57248e-10 m, d_nm = 0.257248
Tetragonal interplanar spacing: a = 0.3 nm, h = 1, k = 0, l = 1	NEEDS_INPUT	d_spacing_tetragonal needs c (lattice parameter c). Example: solve('d_spacing_tetragonal', a='3e-10 m', c='5e-10 m', h=1

Interplanar spacing in an orthorhombic crystal — `d_spacing_orthorhombic`

The spacing of the (h k l) planes of an orthorhombic lattice: $1/d^2 = h^2/a^2 + k^2/b^2 + l^2/c^2$.

$$\frac{1}{d^2} = \frac{h^2}{a^2} + \frac{k^2}{b^2} + \frac{l^2}{c^2}$$

input	meaning	condition
`a`	lattice parameter a	
`b`	lattice parameter b	
`c`	lattice parameter c	
`h`	Miller index h	
`k`	Miller index k	
`l`	Miller index l	

output	meaning
`d`	interplanar spacing
`d_nm`	interplanar spacing in nm

- **Method:** $1/d^2 = h^2/a^2 + k^2/b^2 + l^2/c^2$
- **Assumes:** a, b, $c > 0$; integer indices, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('d_spacing_orthorhombic', a=4e-10, b=5e-10, c=6e-10, h=1, k=1, l=0)
# -> d = 3.12348e-10 m, d_nm = 0.312348   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Orthorhombic interplanar spacing: a = 0.4 nm, b = 0.5 nm, c = 0.6 nm, h = 1, k = 1, l = 0	SOLVED	d = 3.12348e-10 m, d_nm = 0.312348

Interplanar spacing in a hexagonal crystal — `d_spacing_hexagonal`

The spacing of the (h k l) planes of a hexagonal lattice: $1/d^2 = 4$(h^2 + hk + k^2)/(3a^2) + l^2/c^2.

$$\frac{1}{d^2} = \frac{4}{3}\frac{h^2 + hk + k^2}{a^2} + \frac{l^2}{c^2}$$

input	meaning	condition
`a`	lattice parameter a	
`c`	lattice parameter c	
`h`	Miller index h	
`k`	Miller index k	
`l`	Miller index l	
`i`	Miller-Bravais index i = -(h + k) (optional, checked)	

output	meaning
`d`	interplanar spacing
`d_nm`	interplanar spacing in nm

- **Method:** $1/d^2 = 4$(h^2 + hk + k^2)/(3a^2) + l^2/c^2
- **Assumes:** a, $c > 0$; integer indices, not all zero; i = -(h + k) when given
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('d_spacing_hexagonal', a=3.209e-10, c=5.211e-10, h=1, k=0, l=1)
# -> d = 2.45215e-10 m, d_nm = 0.245215   (SOLVED)
```

```
materials_science_ai999x.solve('d_spacing_hexagonal', a=2.5e-10, c=4e-10, h=0, k=0, l=2)
# -> d = 2e-10 m, d_nm = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hexagonal interplanar spacing: a = 0.3209 nm, c = 0.5211 nm, h = 1, k = 0, l = 1	SOLVED	d = 2.45215e-10 m, d_nm = 0.245215
Hexagonal interplanar spacing: a = 0.3209 nm, c = 0.5211 nm, h = 1, k = 0, i = 1, l = 1	INVALID_INPUT	Interplanar spacing in a hexagonal crystal: in (h k i l) the index i must equal -(h + k).

Interplanar spacing in a general (triclinic) lattice — `d_spacing_triclinic`

The spacing of the (h k l) planes of any lattice from its six cell parameters, through the reciprocal metric tensor.

$$\frac{1}{d_{hkl}^2} = \mathbf{h}^{\mathsf{T}} G^{-1} \mathbf{h}$$

input	meaning	condition
`a`	cell edge a	
`b`	cell edge b	
`c`	cell edge c	
`alpha`	cell angle alpha in degrees (between b and c)	
`beta`	cell angle beta in degrees (between a and c)	
`gamma`	cell angle gamma in degrees (between a and b)	
`h`	Miller index h	
`k`	Miller index k	
`l`	Miller index l	

output	meaning
`d`	interplanar spacing
`d_nm`	interplanar spacing in nm

- **Method:** $1/d^2 = h\,Gh^T$ with G* the inverse of the metric tensor $G_ij = a_i$. a_j
- **Assumes:** a valid cell; integer indices, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('d_spacing_triclinic', a=5e-10, b=6e-10, c=7e-10, alpha=90, beta=90,
gamma=90, h=1, k=1, l=0)
# -> d = 3.84111e-10 m, d_nm = 0.384111   (SOLVED)
```

```
materials_science_ai999x.solve('d_spacing_triclinic', a=5e-10, b=5e-10, c=7e-10, alpha=90, beta=90,
gamma=120, h=1, k=0, l=0)
# -> d = 4.33013e-10 m, d_nm = 0.433013   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Triclinic interplanar spacing: a = 0.5 nm, b = 0.5 nm, c = 0.7 nm, alpha = 90, beta = 90, gamma = 120, h = 1, k = 0, l = 0	SOLVED	d = 4.33013e-10 m, d_nm = 0.433013

Bragg angle of a reflection — `bragg_angle`

The diffraction angle theta (and 2-theta) of planes of spacing d for X-rays of wavelength lambda: n $\lambda = 2$ d sin(theta).

$$n\lambda = 2d\sin\theta$$

input	meaning	condition
`d`	interplanar spacing	
`lam`	X-ray wavelength	
`n`	order of the reflection (default 1)	

output	meaning
`theta_deg`	Bragg angle theta in degrees
`two_theta_deg`	diffraction angle $2-\theta$ in degrees

- **Method:** theta = asin(n lambda / (2 d))
- **Assumes:** d, $\lambda > 0$; n $\lambda < 2$ d
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('bragg_angle', d=2.087e-10, lam=1.542e-10)
# -> theta_deg = 21.6805, two_theta_deg = 43.3609   (SOLVED)
```

```
materials_science_ai999x.solve('bragg_angle', d=2e-10, lam=2e-10, n=1)
# -> theta_deg = 30, two_theta_deg = 60   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bragg angle: d = 0.2087 nm, lambda = 0.1542 nm	SOLVED	theta_deg = 21.6805, two_theta_deg = 43.3609
Bragg angle: d = 0.2 nm, lambda = 0.2 nm	SOLVED	theta_deg = 30, two_theta_deg = 60
Bragg angle: d = 0.1 nm, lambda = 0.3 nm	INVALID_INPUT	Bragg angle of a reflection: n lambda must be smaller than 2d (no reflection otherwise).
Bragg angle: d = 0.2 nm	NEEDS_INPUT	bragg_angle needs lam (X-ray wavelength). Example: solve('bragg_angle', d='2.087e-10 m', lam='1.542e-10 m').

X-ray wavelength from a Bragg reflection — `bragg_wavelength`

The wavelength that reflects from planes of spacing d at the Bragg angle theta in order n.

$$\lambda = \frac{2d\sin\theta}{n}$$

input	meaning	condition
`d`	interplanar spacing	
`theta`	Bragg angle theta in degrees (half of 2-theta)	
`n`	order of the reflection (default 1)	

output	meaning
`lam`	wavelength
`lam_nm`	wavelength in nm

- **Method:** $\lambda = 2$ d sin(theta) / n
- **Assumes:** $d > 0$; $0 < \theta < 90$ degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('bragg_wavelength', d=2e-10, theta=30.0)
# -> lam = 2e-10 m, lam_nm = 0.2   (SOLVED)
```

```
materials_science_ai999x.solve('bragg_wavelength', d=3e-10, theta=30.0, n=2)
# -> lam = 1.5e-10 m, lam_nm = 0.15   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bragg wavelength: d = 0.2 nm, theta = 30 deg	SOLVED	lam = 2e-10 m, lam_nm = 0.2
Bragg wavelength: d = 0.2 nm, theta = 95 deg	INVALID_INPUT	X-ray wavelength from a Bragg reflection: the Bragg angle theta must lie between 0 and 90 degrees.

Interplanar spacing from a Bragg reflection — `bragg_spacing`

The interplanar spacing d of planes that reflect X-rays of wavelength lambda at the Bragg angle theta in order n.

$$d = \frac{n\lambda}{2\sin\theta}$$

input	meaning	condition
`lam`	X-ray wavelength	
`theta`	Bragg angle theta in degrees (half of 2-theta)	
`n`	order of the reflection (default 1)	

output	meaning
`d`	interplanar spacing
`d_nm`	interplanar spacing in nm

- **Method:** $d = n$ lambda / (2 sin(theta))
- **Assumes:** $\lambda > 0$; $0 < \theta < 90$ degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('bragg_spacing', lam=1.542e-10, theta=30.0)
# -> d = 1.542e-10 m, d_nm = 0.1542   (SOLVED)
```

```
materials_science_ai999x.solve('bragg_spacing', lam=1e-10, theta=30.0, n=2)
# -> d = 2e-10 m, d_nm = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bragg spacing: lambda = 0.1542 nm, theta = 30 deg	SOLVED	d = 1.542e-10 m, d_nm = 0.1542
Bragg spacing: lambda = 0.1542 nm	NEEDS_INPUT	bragg_spacing needs theta (Bragg angle theta in degrees (half of 2-theta)). Example: solve('bragg_spacing', lam='1.542e-

Diffraction angle of a cubic (h k l) reflection — `cubic_reflection_angle`

The interplanar spacing and the diffraction angle $2-\theta$ of the (h k l) reflection of a cubic crystal of lattice parameter a at wavelength lambda.

$$\sin\theta = \frac{\lambda\sqrt{h^2+k^2+l^2}}{2a}$$

input	meaning	condition
`a`	lattice parameter	
`h`	Miller index h	
`k`	Miller index k	
`l`	Miller index l	
`lam`	X-ray wavelength	

output	meaning
`d_nm`	interplanar spacing in nm
`theta_deg`	Bragg angle in degrees
`two_theta_deg`	diffraction angle $2-\theta$ in degrees

- **Method:** $d = \frac{a}{\sqrt{h^2+k^2+l^2}}$, $\sin(\theta) = \lambda/(2$ d) (first order)
- **Assumes:** a, $\lambda > 0$; $\lambda < 2$ d
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('cubic_reflection_angle', a=3.615e-10, h=1, k=1, l=1, lam=1.542e-10)
# -> d_nm = 0.208712, theta_deg = 21.6791, two_theta_deg = 43.3583   (SOLVED)
```

```
materials_science_ai999x.solve('cubic_reflection_angle', a=4e-10, h=2, k=0, l=0, lam=2e-10)
# -> d_nm = 0.2, theta_deg = 30, two_theta_deg = 60   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cubic reflection angle: a = 0.3615 nm, h = 1, k = 1, l = 1, lambda = 0.1542 nm	SOLVED	d_nm = 0.208712, theta_deg = 21.6791, two_theta_deg = 43.3583

question	outcome	result
Cubic reflection angle: a = 0.3 nm, h = 3, k = 3, l = 3, lambda = 0.2 nm	INVALID_INPUT	Diffraction angle of a cubic (h k l) reflection: this reflection does not occur at this wavelength (lambda >= 2d).

Structure factor and extinction of a cubic reflection — `structure_factor_cubic`

The structure factor |F|/f of the (h k l) reflection of a monatomic SC, BCC, FCC or diamond-cubic crystal, and whether the reflection is allowed.

$$F_{hkl} = f \sum_j e^{2\pi i (h x_j + k y_j + l z_j)}$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC, diamond)	
`h`	Miller index h	
`k`	Miller index k	
`l`	Miller index l	

output	meaning
`F_over_f`	|F| in units of the atomic scattering factor f
`F2_over_f2`	|F|^$2/f^2$ (proportional to intensity)
`allowed`	whether the reflection appears

- **Method:** extinction rules: BCC h+k+l even $(F = 2f)$; FCC unmixed $(F = 4f)$; diamond: all odd (4 sqrt(2) f), all even with $h + k + l = 4m$ (8f), otherwise 0
- **Assumes:** integer indices, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('structure_factor_cubic', structure='FCC', h=1, k=1, l=1)
# -> F_over_f = 4, F2_over_f2 = 16, allowed = True   (SOLVED)
```

```
materials_science_ai999x.solve('structure_factor_cubic', structure='FCC', h=1, k=1, l=0)
# -> F_over_f = 0, F2_over_f2 = 0, allowed = False   (SOLVED)
```

```
materials_science_ai999x.solve('structure_factor_cubic', structure='BCC', h=1, k=1, l=0)
# -> F_over_f = 2, F2_over_f2 = 4, allowed = True   (SOLVED)
```

```
materials_science_ai999x.solve('structure_factor_cubic', structure='diamond', h=2, k=2, l=2)
# -> F_over_f = 0, F2_over_f2 = 0, allowed = False   (SOLVED)
```

```
materials_science_ai999x.solve('structure_factor_cubic', structure='diamond', h=1, k=1, l=1)
# -> F_over_f = 5.65685, F2_over_f2 = 32, allowed = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Structure factor: structure = FCC, h = 2, k = 0, l = 0	SOLVED	F_over_f = 4, F2_over_f2 = 16, allowed = True
Structure factor: structure = BCC, h = 1, k = 0, l = 0	SOLVED	F_over_f = 0, F2_over_f2 = 0, allowed = False
Structure factor: structure = FCC, h = 0, k = 0, l = 0	INVALID_INPUT	Structure factor and extinction of a cubic reflection: the Miller indices must not all be zero.

First allowed reflections of a cubic crystal — `allowed_reflections_cubic`

The values of $h^2 + k^2 + l^2$ of the first allowed X-ray reflections of a monatomic SC, BCC, FCC or diamond-cubic crystal, in order of increasing angle.

$$N = h^2 + k^2 + l^2, \quad \sin^2\theta = \frac{\lambda^2}{4a^2} N$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC, diamond)	
`count`	how many reflections (1 to 15, default 6)	

output	meaning
`N_values`	$h^2 + k^2 + l^2$ of the allowed reflections, ascending
`N_last`	the last of them
`N_first`	the first allowed reflection

- **Method:** each $N = h^2 + k^2 + l^2$ is allowed when some (h k l) with that sum passes the extinction rules
- **Assumes:** 1 <= count <= 15
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('allowed_reflections_cubic', structure='BCC', count=6)
# -> N_values = [2, 4, 6, 8, 10, 12], N_last = 12, N_first = 2   (SOLVED)
```

```
materials_science_ai999x.solve('allowed_reflections_cubic', structure='FCC', count=6)
# -> N_values = [3, 4, 8, 11, 12, 16], N_last = 16, N_first = 3   (SOLVED)
```

```
materials_science_ai999x.solve('allowed_reflections_cubic', structure='SC', count=7)
# -> N_values = [1, 2, 3, 4, 5, 6, 8], N_last = 8, N_first = 1   (SOLVED)
```

```
materials_science_ai999x.solve('allowed_reflections_cubic', structure='diamond', count=5)
# -> N_values = [3, 8, 11, 16, 19], N_last = 19, N_first = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Allowed reflections: structure = FCC, count = 6	SOLVED	N_values = [3, 4, 8, 11, 12, 16], N_last = 16, N_first = 3
Allowed reflections: structure = BCC	SOLVED	N_values = [2, 4, 6, 8, 10, 12], N_last = 12, N_first = 2

question	outcome	result
Allowed reflections: structure = FCC, count = 40	INVALID_INPUT	count: must be at most 15, got 40.

Linear density along a crystal direction — `linear_density`

The number of atoms per unit length along the [u v w] direction of an SC, BCC or FCC crystal, and the linear packing fraction.

$$\mathrm{LD} = \frac{\text{atoms centred on the line}}{\text{length of the line}}$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC)	
`u`	direction index u	
`v`	direction index v	
`w`	direction index w	
`a`	lattice parameter	

output	meaning
`LD_per_nm`	atoms per nm along [uvw]
`LD`	atoms per metre along [uvw]
`repeat_nm`	repeat distance along [uvw] in nm
`LPF`	linear packing fraction (hard spheres)

- **Method:** the shortest lattice translation along [uvw]: |[uvw]| a, halved when the half vector is a lattice vector (BCC all odd, FCC two odd)
- **Assumes:** $a > 0$; integer indices between -4 and 4, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('linear_density', structure='FCC', u=1, v=1, w=0, a=3.615e-10)
# -> LD_per_nm = 3.91207, LD = 3.91207e+09 1/m, repeat_nm = 0.255619, LPF = 1   (SOLVED)
```

```
materials_science_ai999x.solve('linear_density', structure='BCC', u=1, v=1, w=1, a=2.866e-10)
# -> LD_per_nm = 4.02896, LD = 4.02896e+09 1/m, repeat_nm = 0.248203, LPF = 1   (SOLVED)
```

```
materials_science_ai999x.solve('linear_density', structure='FCC', u=1, v=0, w=0, a=4e-10)
# -> LD_per_nm = 2.5, LD = 2.5e+09 1/m, repeat_nm = 0.4, LPF = 0.707107   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Linear density: structure = FCC, u = 1, v = 1, w = 0, a = 0.3615 nm	SOLVED	LD_per_nm = 3.91207, LD = 3.91207e+09 1/m, repeat_nm = 0.255619, LPF = 1
Linear density: structure = BCC, u = 1, v = 0, w = 0, a = 0.2866 nm	SOLVED	LD_per_nm = 3.48918, LD = 3.48918e+09 1/m, repeat_nm = 0.2866, LPF = 0.866025

question	outcome	result
Linear density: structure = FCC, u = 0, v = 0, w = 0, a = 0.3615 nm	INVALID_INPUT	Linear density along a crystal direction: the direction indices must not all be zero.

Planar density of a crystal plane — `planar_density`

The number of atoms per unit area on the (h k l) plane of an SC, BCC or FCC crystal, and the planar packing fraction.

$$\mathrm{PD} = n_V \, d_{\mathrm{planes}}, \quad \mathrm{PPF} = \mathrm{PD}\,\pi R^2$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC)	
`h`	Miller index h	
`k`	Miller index k	
`l`	Miller index l	
`a`	lattice parameter	

output	meaning
`PD_per_nm2`	atoms per nm^2 on the plane
`PD`	atoms per m^2 on the plane
`PPF`	planar packing fraction (hard spheres)

- **Method:** PD = (atoms per volume) x (spacing of the lattice planes parallel to (hkl)); the lattice-plane spacing is a/sqrt(N), halved when (hkl) is not a reciprocal-lattice vector (BCC h+k+l odd, FCC mixed)
- **Assumes:** $a > 0$; integer indices between -3 and 3, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('planar_density', structure='FCC', h=1, k=0, l=0, a=4e-10)
# -> PD_per_nm2 = 12.5, PD = 1.25e+19 1/m^2, PPF = 0.785398   (SOLVED)
```

```
materials_science_ai999x.solve('planar_density', structure='FCC', h=1, k=1, l=1, a=4e-10)
# -> PD_per_nm2 = 14.4338, PD = 1.44338e+19 1/m^2, PPF = 0.9069   (SOLVED)
```

```
materials_science_ai999x.solve('planar_density', structure='BCC', h=1, k=1, l=0, a=3e-10)
# -> PD_per_nm2 = 15.7135, PD = 1.57135e+19 1/m^2, PPF = 0.833041   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Planar density: structure = FCC, h = 1, k = 1, l = 1, a = 0.4 nm	SOLVED	PD_per_nm2 = 14.4338, PD = 1.44338e+19 1/m^2, PPF = 0.9069
Planar density: structure = BCC, h = 1, k = 1, l = 0, a = 0.3 nm	SOLVED	PD_per_nm2 = 15.7135, PD = 1.57135e+19 1/m^2, PPF = 0.833041

question	outcome	result
Planar density: structure = FCC, h = 1, k = 1, l = 1	NEEDS_INPUT	planar_density needs a (lattice parameter). Example: solve('planar_density', structure='FCC', h=1, k=0, l=0, a='4e-10 m'

Miller indices of a plane from its intercepts — `miller_indices_from_intercepts`

The Miller indices (h k l) of a plane cutting the axes at x, y, z (in units of the cell edges; inf for a plane parallel to an axis).

$$(hkl) \propto \left(\frac{1}{x}, \frac{1}{y}, \frac{1}{z}\right)$$

input	meaning	condition
`x`	intercept on the a axis (number, fraction or inf)	
`y`	intercept on the b axis	
`z`	intercept on the c axis	

output	meaning
`h`	Miller index h
`k`	Miller index k
`l`	Miller index l
`indices`	(h k l)

- **Method:** take reciprocals of the intercepts, clear fractions, reduce to the smallest integers (signs kept)
- **Assumes:** no intercept 0 (move the origin); at least one finite intercept
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('miller_indices_from_intercepts', x='1', y='1/2', z='inf')
# -> h = 1, k = 2, l = 0, indices = (1 2 0)   (SOLVED)
```

```
materials_science_ai999x.solve('miller_indices_from_intercepts', x='1/2', y='1', z='2/3')
# -> h = 4, k = 2, l = 3, indices = (4 2 3)   (SOLVED)
```

```
materials_science_ai999x.solve('miller_indices_from_intercepts', x='-1', y='inf', z='1/2')
# -> h = -1, k = 0, l = 2, indices = (-1 0 2)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Miller indices from intercepts: x = 1, y = 1/2, z = inf	SOLVED	h = 1, k = 2, l = 0, indices = (1 2 0)
Miller indices from intercepts: x = 0, y = 1, z = 1	INVALID_INPUT	Miller indices of a plane from its intercepts: an intercept of 0 means the plane passes through the origin: move the ori
Miller indices from intercepts: x = 1, y = 1	NEEDS_INPUT	miller_indices_from_intercepts needs z (intercept on the c axis). Example: solve('miller_indices_from_intercepts', x='1'

Three-index to four-index (Miller-Bravais) direction — `miller_bravais_direction`

Convert a hexagonal direction [U V W] (axes a1, a2, c) to the Miller-Bravais form [u v t w] (axes a1, a2, a3, c).

$$u = \tfrac{1}{3}(2U - V),\ v = \tfrac{1}{3}(2V - U),\ t = -(u + v),\ w = W$$

input	meaning	condition
`U`	index along a1	
`V`	index along a2	
`W`	index along c	

output	meaning
`u`	Miller-Bravais index u
`v`	Miller-Bravais index v
`t`	Miller-Bravais index t
`w`	Miller-Bravais index w
`indices`	[u v t w]

- **Method:** $u = (2U - V)/3$, $v = (2V - U)/3$, t = -(u + v), $w = W$, reduced to the smallest integers
- **Assumes:** integer indices, not all zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('miller_bravais_direction', U=1, V=0, W=0)
# -> u = 2, v = -1, t = -1, w = 0, indices = [2 -1 -1 0]   (SOLVED)
```

```
materials_science_ai999x.solve('miller_bravais_direction', U=0, V=0, W=1)
# -> u = 0, v = 0, t = 0, w = 1, indices = [0 0 0 1]   (SOLVED)
```

```
materials_science_ai999x.solve('miller_bravais_direction', U=1, V=1, W=0)
# -> u = 1, v = 1, t = -2, w = 0, indices = [1 1 -2 0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Miller-Bravais direction: U = 1, V = 0, W = 0	SOLVED	u = 2, v = -1, t = -1, w = 0, indices = [2 -1 -1 0]
Miller-Bravais direction: U = 0, V = 0, W = 0	INVALID_INPUT	Three-index to four-index (Miller-Bravais) direction: the direction indices must not all be zero.

Four-index (Miller-Bravais) to three-index direction — `miller_bravais_to_three_index`

Convert a Miller-Bravais direction [u v t w] of a hexagonal crystal to the three-index form [U V W].

$$U = u - t,\ V = v - t,\ W = w$$

input	meaning	condition
`u`	index along a1	
`v`	index along a2	
`t`	index along a3 (= -(u + v))	
`w`	index along c	

output	meaning
`U`	three-index component U
`V`	three-index component V
`W`	three-index component W
`indices`	[U V W]

- **Method:** $U = u - t$, $V = v - t$, $W = w$, reduced to the smallest integers
- **Assumes:** integer indices; t = -(u + v)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('miller_bravais_to_three_index', u=2, v=-1, t=-1, w=0)
# -> U = 1, V = 0, W = 0, indices = [1 0 0]   (SOLVED)
```

```
materials_science_ai999x.solve('miller_bravais_to_three_index', u=1, v=1, t=-2, w=3)
# -> U = 1, V = 1, W = 1, indices = [1 1 1]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Four to three index direction: u = 2, v = -1, t = -1, w = 0	SOLVED	U = 1, V = 0, W = 0, indices = [1 0 0]
Four to three index direction: u = 1, v = 1, t = 1, w = 0	INVALID_INPUT	Four-index (Miller-Bravais) to three-index direction: in [u v t w] the index t must equal -(u + v).

Angle between two directions in a cubic crystal — `angle_between_directions_cubic`

The angle between the directions [u1 v1 w1] and [u2 v2 w2] of a cubic crystal (also the angle between the planes with those indices).

$$\cos\varphi = \frac{u_1u_2 + v_1v_2 + w_1w_2}{\sqrt{u_1^2 + v_1^2 + w_1^2}\sqrt{u_2^2 + v_2^2 + w_2^2}}$$

input	meaning	condition
`u1`	index u of direction 1	
`v1`	index v of direction 1	
`w1`	index w of direction 1	
`u2`	index u of direction 2	
`v2`	index v of direction 2	

input	meaning	condition
`w2`	index w of direction 2	

output	meaning
`angle_deg`	angle in degrees

- **Method:** cos(angle) = (u1 $u_2 + v_1 v_2 + w_1$ w2) / (|d1| |d2|), evaluated as atan2(|d1 x d2|, d1 . d2)
- **Assumes:** integer indices, neither direction zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('angle_between_directions_cubic', u1=1, v1=0, w1=0, u2=1, v2=1,
w2=1)
# -> angle_deg = 54.7356   (SOLVED)
```

```
materials_science_ai999x.solve('angle_between_directions_cubic', u1=1, v1=1, w1=0, u2=1, v2=-1,
w2=0)
# -> angle_deg = 90   (SOLVED)
```

```
materials_science_ai999x.solve('angle_between_directions_cubic', u1=1, v1=1, w1=1, u2=-1, v2=-1,
w2=-1)
# -> angle_deg = 180   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Angle between cubic directions: u1 = 1, v1 = 0, w1 = 0, u2 = 1, v2 = 1, w2 = 1	SOLVED	angle_deg = 54.7356
Angle between cubic directions: u1 = 0, v1 = 0, w1 = 0, u2 = 1, v2 = 1, w2 = 1	INVALID_INPUT	Angle between two directions in a cubic crystal: the first direction must not all be zero.

Angle between two lattice directions in any crystal — `angle_between_directions`

The angle between the directions [u1 v1 w1] and [u2 v2 w2] of a crystal with any cell (a, b, c, alpha, beta, gamma).

$$\cos\varphi = \frac{\mathbf{d}_1^{\mathsf{T}} G\, \mathbf{d}_2}{\sqrt{\mathbf{d}_1^{\mathsf{T}} G\, \mathbf{d}_1}\sqrt{\mathbf{d}_2^{\mathsf{T}} G\, \mathbf{d}_2}}$$

input	meaning	condition
`a`	cell edge a	
`b`	cell edge b	
`c`	cell edge c	
`alpha`	cell angle alpha in degrees (between b and c)	
`beta`	cell angle beta in degrees (between a and c)	
`gamma`	cell angle gamma in degrees (between a and b)	

input	meaning	condition
`u1`	index u of direction 1	
`v1`	index v of direction 1	
`w1`	index w of direction 1	
`u2`	index u of direction 2	
`v2`	index v of direction 2	
`w2`	index w of direction 2	

output	meaning
`angle_deg`	angle in degrees

- **Method:** the metric tensor G: cos(angle) = d1 G $d_2/sqrt$((d1 G d1)(d2 G d2))
- **Assumes:** a valid cell; neither direction zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('angle_between_directions', a=3e-10, b=3e-10, c=5e-10, alpha=90,
beta=90, gamma=120, u1=1, v1=0, w1=0, u2=0, v2=1, w2=0)
# -> angle_deg = 120   (SOLVED)
```

```
materials_science_ai999x.solve('angle_between_directions', a=3e-10, b=4e-10, c=5e-10, alpha=90,
beta=90, gamma=90, u1=1, v1=1, w1=0, u2=1, v2=0, w2=0)
# -> angle_deg = 53.1301   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Angle between lattice directions: a = 0.3 nm, b = 0.3 nm, c = 0.5 nm, alpha = 90, beta = 90, gamma = 120, u1 = 1, v1 = 0, w1 = 0, u2 = 0, v2 = 1, w2 = 0	SOLVED	angle_deg = 120

Angle between two lattice planes in any crystal — `angle_between_planes`

The angle between the normals of the planes (h1 k1 l1) and (h2 k2 l2) of a crystal with any cell.

$$\cos\varphi = \frac{\mathbf{h}_1^{\mathsf{T}} G^{-1} \mathbf{h}_2}{\sqrt{\mathbf{h}_1^{\mathsf{T}} G^{-1} \mathbf{h}_1}\sqrt{\mathbf{h}_2^{\mathsf{T}} G^{-1} \mathbf{h}_2}}$$

input	meaning	condition
`a`	cell edge a	
`b`	cell edge b	
`c`	cell edge c	
`alpha`	cell angle alpha in degrees (between b and c)	
`beta`	cell angle beta in degrees (between a and c)	
`gamma`	cell angle gamma in degrees (between a and b)	
`h1`	Miller index h of plane 1	

input	meaning	condition
`k1`	Miller index k of plane 1	
`l1`	Miller index l of plane 1	
`h2`	Miller index h of plane 2	
`k2`	Miller index k of plane 2	
`l2`	Miller index l of plane 2	

output	meaning
`angle_deg`	angle between the plane normals in degrees

- **Method:** the reciprocal metric tensor G* = G^{-1}: cos(angle) = h1 $Gh_2/sqrt$((h1 G* h1)(h2 G* h2))
- **Assumes:** a valid cell; neither plane (0 0 0)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('angle_between_planes', a=4e-10, b=4e-10, c=4e-10, alpha=90,
beta=90, gamma=90, h1=1, k1=1, l1=1, h2=1, k2=1, l2=-1)
# -> angle_deg = 70.5288   (SOLVED)
```

```
materials_science_ai999x.solve('angle_between_planes', a=3e-10, b=3e-10, c=5e-10, alpha=90,
beta=90, gamma=120, h1=1, k1=0, l1=0, h2=0, k2=1, l2=0)
# -> angle_deg = 60   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Angle between planes: a = 0.4 nm, b = 0.4 nm, c = 0.4 nm, alpha = 90, beta = 90, gamma = 90, h1 = 1, k1 = 1, l1 = 1, h2 = 1, k2 = 1, l2 = -1	SOLVED	angle_deg = 70.5288

Largest atom that fits an interstitial site — `interstitial_site_size`

The radius of the largest sphere that fits the octahedral or tetrahedral interstitial site of an FCC or BCC lattice of host atoms of radius R.

$$r = d_{\text{site}\to\text{atom}} - R$$

input	meaning	condition
`structure`	crystal structure (FCC, BCC)	
`site`	interstitial site (octahedral, tetrahedral)	
`R`	radius of the host atoms	

output	meaning
`r`	radius of the interstitial sphere
`r_nm`	interstitial radius in nm
`ratio`	r/R

- **Method:** hard spheres: distance from the site centre to the nearest host atom minus R (FCC oct $sqrt2 - 1$, tet sqrt(3/2)-1; BCC oct $2/sqrt3 - 1$, tet sqrt(5/3)-1)
- **Assumes:** $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('interstitial_site_size', structure='FCC', site='octahedral',
R=1.29e-10)
# -> r = 5.34335e-11 m, r_nm = 0.0534335, ratio = 0.414214   (SOLVED)
```

```
materials_science_ai999x.solve('interstitial_site_size', structure='BCC', site='tetrahedral',
R=1.24e-10)
# -> r = 3.60833e-11 m, r_nm = 0.0360833, ratio = 0.290994   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Interstitial site size: structure = FCC, site = octahedral, R = 0.129 nm	SOLVED	r = 5.34335e-11 m, r_nm = 0.0534335, ratio = 0.414214
Interstitial site size: structure = BCC, site = cubic, R = 0.129 nm	INVALID_INPUT	site: one of octahedral, tetrahedral; got 'cubic'.

Coordination number from the cation-anion radius ratio — `radius_ratio_coordination`

The expected coordination number and polyhedron of a cation from the ratio of the cation and anion radii (Pauling's radius-ratio rule).

$$\frac{r_c}{r_a}:\ 0.155\ (3),\ 0.225\ (4),\ 0.414\ (6),\ 0.732\ (8)$$

input	meaning	condition
`rc`	cation radius	
`ra`	anion radius	

output	meaning
`ratio`	rc/ra
`CN`	coordination number
`geometry`	coordination polyhedron

- **Method:** critical ratios 0.155 (3), 0.225 (4), 0.414 (6), 0.732 (8): the largest one not above rc/ra
- **Assumes:** $0 < rc < ra$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('radius_ratio_coordination', rc=1.02e-10, ra=1.81e-10)
# -> ratio = 0.563536, CN = 6, geometry = octahedral   (SOLVED)
```

```
materials_science_ai999x.solve('radius_ratio_coordination', rc=1.7e-10, ra=1.81e-10)
# -> ratio = 0.939227, CN = 8, geometry = cubic   (SOLVED)
```

```
materials_science_ai999x.solve('radius_ratio_coordination', rc=4e-11, ra=1.4e-10)
# -> ratio = 0.285714, CN = 4, geometry = tetrahedral   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radius ratio: rc = 0.102 nm, ra = 0.181 nm	SOLVED	ratio = 0.563536, CN = 6, geometry = octahedral
Radius ratio: rc = 0.2 nm, ra = 0.1 nm	INVALID_INPUT	Coordination number from the cation-anion radius ratio: the cation must be smaller than the anion (rc/ra < 1) for the ra

Lattice parameter and density of an ionic crystal — `ceramic_density`

The cubic lattice parameter and theoretical density of a rock-salt, cesium-chloride, zinc-blende or fluorite crystal from its ionic radii and molar masses.

$$\rho = \frac{n' \left(\sum A_C + \sum A_A\right)}{V_C N_A}$$

input	meaning	condition
`prototype`	crystal structure (rocksalt, CsCl, zincblende, fluorite)	
`rc`	cation radius	
`ra`	anion radius	
`Ac`	molar mass of the cation element	
`Aa`	molar mass of the anion element	

output	meaning
`a`	lattice parameter
`a_nm`	lattice parameter in nm
`rho`	density
`rho_gcc`	density in g/cm^3

- **Method:** cation-anion contact: rock salt $a = 2$(rc + ra), CsCl $a = 2$(rc + ra)/sqrt3, zinc blende and fluorite $a = 4$(rc + ra)/sqrt3; $\rho = n$'(Ac + k Aa)/(a^3 N_A)
- **Assumes:** radii and molar masses > 0; cations and anions touch (the anions need not)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('ceramic_density', prototype='rocksalt', rc=1.02e-10, ra=1.81e-10,
Ac=0.02299, Aa=0.03545)
# -> a = 5.66e-10 m, a_nm = 0.566, rho = 2140.77 kg/m^3, rho_gcc = 2.14077   (SOLVED)
```

```
materials_science_ai999x.solve('ceramic_density', prototype='CsCl', rc=1.7e-10, ra=1.81e-10,
Ac=0.13291, Aa=0.03545)
# -> a = 4.053e-10 m, a_nm = 0.4053, rho = 4199.12 kg/m^3, rho_gcc = 4.19912   (SOLVED)
```

```
materials_science_ai999x.solve('ceramic_density', prototype='fluorite', rc=1e-10, ra=1.33e-10,
Ac=0.04008, Aa=0.019)
# -> a = 5.3809e-10 m, a_nm = 0.53809, rho = 3328.76 kg/m^3, rho_gcc = 3.32876   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ceramic density: prototype = rocksalt, rc = 0.102 nm, ra = 0.181 nm, Ac = 22.99 g/mol, Aa = 35.45 g/mol	SOLVED	a = 5.66e-10 m, a_nm = 0.566, rho = 2140.77 kg/m^3, rho_gcc = 2.14077
Ceramic density: prototype = perovskite, rc = 0.102 nm, ra = 0.181 nm, Ac = 22.99 g/mol, Aa = 35.45 g/mol	INVALID_INPUT	prototype: one of rocksalt, CsCl, zincblende, fluorite; got 'perovskite'.

Volume change of a BCC to FCC transformation — `polymorphic_volume_change`

The percentage volume change when a metal transforms from BCC to FCC (as iron at 912 C), from the atomic radius in each structure.

$$\frac{\Delta V}{V} = \frac{a_F^3/4 - a_B^3/2}{a_B^3/2}$$

input	meaning	condition
`R_bcc`	atomic radius in the BCC form	
`R_fcc`	atomic radius in the FCC form	

output	meaning
`volume_change_percent`	volume change BCC -> FCC in percent (negative = contraction)
`V_atom_bcc_nm3`	volume per atom in BCC, nm^3
`V_atom_fcc_nm3`	volume per atom in FCC, nm^3

- **Method:** volume per atom: BCC $(4R/sqrt3)^3/2$, FCC (2 sqrt2 R)^3/4
- **Assumes:** radii > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('polymorphic_volume_change', R_bcc=1.258e-10, R_fcc=1.292e-10)
# -> volume_change_percent = -0.493257, V_atom_bcc_nm3 = 0.0122606, V_atom_fcc_nm3 = 0.0122001
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
BCC to FCC volume change: R_bcc = 0.1258 nm, R_fcc = 0.1292 nm	SOLVED	volume_change_percent = -0.493257, V_atom_bcc_nm3 = 0.0122606, V_atom_fcc_nm3 = 0.0122001

question	outcome	result
BCC to FCC volume change: R_bcc = 0.1258 nm	NEEDS_INPUT	polymorphic_volume_change needs R_fcc (atomic radius in the FCC form). Example: solve('polymorphic_volume_change', R_bcc

Burgers vector of a perfect dislocation — `burgers_vector`

The magnitude of the shortest lattice translation (the Burgers vector of a perfect dislocation) in an SC, BCC or FCC metal.

$$b_{\mathrm{FCC}} = \frac{a}{\sqrt{2}},\ b_{\mathrm{BCC}} = \frac{a\sqrt{3}}{2},\ b_{\mathrm{SC}} = a$$

input	meaning	condition
`structure`	crystal structure (SC, BCC, FCC)	
`a`	lattice parameter	

output	meaning
`b`	Burgers vector magnitude
`b_nm`	Burgers vector in nm
`direction`	Burgers vector family

- **Method:** shortest lattice translation: SC a < 100>, BCC $(a/2)$ < 111> = a $sqrt3/2$, FCC $(a/2)$ < 110> = $a/sqrt2$
- **Assumes:** $a > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('burgers_vector', structure='FCC', a=3.615e-10)
# -> b = 2.55619e-10 m, b_nm = 0.255619, direction = a/2<110>   (SOLVED)
```

```
materials_science_ai999x.solve('burgers_vector', structure='BCC', a=2.866e-10)
# -> b = 2.48203e-10 m, b_nm = 0.248203, direction = a/2<111>   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Burgers vector: structure = FCC, a = 0.3615 nm	SOLVED	b = 2.55619e-10 m, b_nm = 0.255619, direction = a/2<110>
Burgers vector: structure = FCC	NEEDS_INPUT	burgers_vector needs a (lattice parameter). Example: solve('burgers_vector', structure='FCC', a='3.615e-10 m').

Space group of a common crystal structure — `space_group`

The space-group number, Hermann-Mauguin symbol and crystal system of a common metallic or ionic structure.

$$\text{e.g. FCC: } Fm\bar{3}m\ (225)$$

input	meaning	condition
`structure`	structure (SC, BCC, FCC, HCP, diamond, rocksalt, CsCl, zincblende, fluorite)	

output	meaning
`number`	space-group number (International Tables)
`symbol`	Hermann-Mauguin symbol
`crystal_system`	crystal system

- **Method:** the symmetry of the ideal structure (International Tables numbering)
- **Assumes:**
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('space_group', structure='diamond')
# -> number = 227, symbol = Fd-3m, crystal_system = cubic   (SOLVED)
```

```
materials_science_ai999x.solve('space_group', structure='HCP')
# -> number = 194, symbol = P6_3/mmc, crystal_system = hexagonal   (SOLVED)
```

```
materials_science_ai999x.solve('space_group', structure='zincblende')
# -> number = 216, symbol = F-43m, crystal_system = cubic   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Space group: structure = BCC	SOLVED	number = 229, symbol = Im-3m, crystal_system = cubic
Space group: structure = quasicrystal	INVALID_INPUT	structure: one of SC, BCC, FCC, HCP, diamond, rocksalt, CsCl, zincblende, fluorite; got 'quasicrystal'.

Equilibrium vacancy concentration — `vacancy_concentration`

The number of atomic sites per volume and the equilibrium number of vacancies $N_v = N$ exp(-Q_v/kT) of a metal.

$$N_v = N \exp\left(-\frac{Q_v}{kT}\right), \quad N = \frac{N_A \rho}{A}$$

input	meaning	condition
`rho`	density	
`A`	molar mass	
`Qv`	vacancy formation energy (per vacancy, e.g. eV)	
`T`	temperature	

output	meaning
`N`	atomic sites per volume
`Nv`	vacancies per volume

output	meaning
`fraction`	vacancy fraction Nv/N
`log10_Nv`	log10 of Nv in m^{-3}
`N_per_cm3`	atomic sites per cm^3

- **Method:** $N = N_A$ rho / A; $N_v = N$ exp(-Q_v / (k T))
- **Assumes:** all inputs > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('vacancy_concentration', rho=8400.0, A=0.0635, Qv=1.4419589706e-19,
T=1000.0)
# -> N = 7.9663e+28 1/m^3, Nv = 2.31983e+24 1/m^3, fraction = 2.91206e-05, log10_Nv = 24.3655,
N_per_cm3 = 7.9663e+22   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Vacancy concentration: rho = 8.4 g/cm^3, A = 63.5 g/mol, Qv = 0.9 eV, T = 1000 K	SOLVED	N = 7.9663e+28 1/m^3, Nv = 2.31983e+24 1/m^3, fraction = 2.91206e-05, log10_Nv = 24.3655, N_per_cm3 = 7.9663e+22
Vacancy concentration: rho = 8.4 g/cm^3, A = 63.5 g/mol, Qv = 0.9 eV	NEEDS_INPUT	vacancy_concentration needs T (temperature). Example: solve('vacancy_concentration', rho='8400 kg/m^3', A='0.0635 kg/mol

Vacancy formation energy from vacancy fractions — `vacancy_formation_energy_fit`

The vacancy formation energy Q_v and prefactor from vacancy fractions measured at two or more temperatures (least-squares Arrhenius line).

$$\ln \frac{N_v}{N} = \ln C - \frac{Q_v}{k}\frac{1}{T}$$

input	meaning	condition
`T`	temperatures in kelvin	
`fraction`	vacancy fractions N_v/N at those temperatures	

output	meaning
`Qv_eV`	vacancy formation energy in eV
`Qv`	vacancy formation energy
`prefactor`	prefactor C of $N_v/N = C$ exp(-Q_v/kT)
`r_squared`	r^2 of the fit

- **Method:** least-squares line of ln(N_v/N) against $1/T$: slope = -Q_v/k
- **Assumes:** equal lengths, at least two distinct temperatures; 0 < fraction < 1; fraction rising with T
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('vacancy_formation_energy_fit', T=[800.0, 1000.0], fraction=[1.2e-06, 3.4e-05])
# -> Qv_eV = 1.15267, Qv = 1.84678e-19 J, prefactor = 21.9114, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Vacancy formation energy: T = [800, 1000], fraction = [1.2e-6, 3.4e-5]	SOLVED	Qv_eV = 1.15267, Qv = 1.84678e-19 J, prefactor = 21.9114, r_squared = 1
Vacancy formation energy: T = [800, 1000], fraction = [3.4e-5, 1.2e-6]	INVALID_INPUT	Vacancy formation energy from vacancy fractions: the vacancy fraction must increase with temperature.
Vacancy formation energy: T = [800, 1000]	NEEDS_INPUT	vacancy_formation_energy_fit needs fraction (vacancy fractions N_v/N at those temperatures). Example: solve('vacancy_for

Steady-state diffusion flux (Fick's first law) — `fick_first_law_flux`

The steady-state diffusion flux between two positions with known concentrations: $J = -D\,(C_2 - C_1)/(x_2 - x_1)$.

$$J = -D\frac{dC}{dx} = -D\frac{C_2 - C_1}{x_2 - x_1}$$

input	meaning	condition
`D`	diffusion coefficient	
`C1`	concentration at x1	
`C2`	concentration at x2	
`x1`	first position	
`x2`	second position	

output	meaning
`J`	diffusion flux (positive towards increasing x)
`J_per_hour`	flux in kg/(m^2 h)

- **Method:** $J = -D$ dC/dx with the linear steady-state profile
- **Assumes:** $D > 0$; $x_1 \neq x_2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('fick_first_law_flux', D=3e-11, C1=1.2, C2=0.8, x1=0.005, x2=0.01)
# -> J = 2.4e-09 kg/(m^2*s), J_per_hour = 8.64e-06   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fick's first law: D = 3e-11 m^2/s, C1 = 1.2 kg/m^3, C2 = 0.8 kg/m^3, x1 = 5 mm, x2 = 10 mm	SOLVED	J = 2.4e-09 kg/(m^2*s), J_per_hour = 8.64e-06

question	outcome	result
Fick's first law: D = 3e-11 m^2/s, C1 = 1.2 kg/m^3, C2 = 0.8 kg/m^3, x1 = 5 mm, x2 = 5 mm	INVALID_INPUT	Steady-state diffusion flux (Fick's first law): the two positions must differ.

Concentration in a semi-infinite solid (Fick's second law) — `fick_second_law_concentration`

The concentration at depth x after time t in a semi-infinite solid with a constant surface concentration (carburizing, doping): $C_x = C_s - (C_s - C_0)$ erf($x/2$ sqrt(Dt)).

$$\frac{C_x - C_0}{C_s - C_0} = 1 - \operatorname{erf}\left(\frac{x}{2\sqrt{Dt}}\right)$$

input	meaning	condition
`C0`	initial (bulk) concentration (wt%, at% or any one unit)	
`Cs`	surface concentration (same unit)	
`x`	depth below the surface	
`D`	diffusion coefficient	
`t`	diffusion time	

output	meaning
`Cx`	concentration at depth x (unit of C0, Cs)
`z`	erf argument x / (2 sqrt(D t))
`ratio`	$(Cx - C_0)/(Cs - C_0)$

- **Method:** the error-function solution of $dC/dt = D\ d2C/dx2$
- **Assumes:** x, D, $t > 0$; $Cs \neq C_0$; constant D, surface concentration held constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('fick_second_law_concentration', C0=0.25, Cs=1.2, x=0.0005, D=1.6e-11, t=25200.0)
# -> Cx = 0.798785, z = 0.393713, ratio = 0.577668   (SOLVED)
```

```
materials_science_ai999x.solve('fick_second_law_concentration', C0=0.0, Cs=1.0, x=0.0002, D=1e-10, t=100.0)
# -> Cx = 0.157299, z = 1, ratio = 0.157299   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fick's second law: C0 = 0.25 wt%, Cs = 1.2 wt%, x = 0.5 mm, D = 1.6e-11 m^2/s, t = 7 h	SOLVED	Cx = 0.798785, z = 0.393713, ratio = 0.577668
Fick's second law: C0 = 0.25 wt%, Cs = 0.25 wt%, x = 0.5 mm, D = 1.6e-11 m^2/s, t = 7 h	INVALID_INPUT	Concentration in a semi-infinite solid (Fick's second law): the surface and initial concentrations must differ.

Time to reach a concentration at a depth (Fick's second law) —

`diffusion_time_to_concentration`

The diffusion time for the concentration at depth x to reach C_x in a semi-infinite solid with constant surface concentration (carburizing time).

$$t = \frac{x^2}{4Dz^2}, \quad \operatorname{erf} z = \frac{C_s - C_x}{C_s - C_0}$$

input	meaning	condition
`C0`	initial concentration	
`Cs`	surface concentration	
`Cx`	target concentration at depth x	
`x`	depth	
`D`	diffusion coefficient	

output	meaning
`t`	diffusion time
`t_hours`	diffusion time in hours
`z`	erf argument

- **Method:** erf(z) = (Cs - Cx)/(Cs - C0) inverted by bisection; t = x^2/(4 D z^2)
- **Assumes:** x, $D > 0$; Cx strictly between C0 and Cs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('diffusion_time_to_concentration', C0=0.0, Cs=1.0,
Cx=0.4795001221869535, x=0.001, D=1e-10)
# -> t = 10000 s, t_hours = 2.77778, z = 0.5   (SOLVED)
```

```
materials_science_ai999x.solve('diffusion_time_to_concentration', C0=0.25, Cs=1.2, Cx=0.8,
x=0.0005, D=1.6e-11)
# -> t = 25370.2 s, t_hours = 7.04729, z = 0.39239   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diffusion time: C0 = 0, Cs = 1, Cx = 0.4795001221869535, x = 1 mm, D = 1e-10 m^2/s	SOLVED	t = 10000 s, t_hours = 2.77778, z = 0.5
Diffusion time: C0 = 0.25 wt%, Cs = 1.2 wt%, Cx = 1.5 wt%, x = 0.5 mm, D = 1.6e-11 m^2/s	INVALID_INPUT	Time to reach a concentration at a depth (Fick's second law): the target concentration must lie strictly between C0 and

Depth at which a concentration is reached (Fick's second law) — `diffusion_depth_of_concentration`

The depth at which the concentration equals C_x after diffusion time t in a semi-infinite solid with constant surface concentration (case depth).

$$x = 2z\sqrt{Dt}$$

input	meaning	condition
`C0`	initial concentration	
`Cs`	surface concentration	
`Cx`	target concentration	
`D`	diffusion coefficient	
`t`	diffusion time	

output	meaning
`x`	depth
`x_mm`	depth in mm
`z`	erf argument

- **Method:** $x = 2$ z sqrt(D t) with erf(z) = (Cs - Cx)/(Cs - C0)
- **Assumes:** D, $t > 0$; Cx strictly between C0 and Cs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('diffusion_depth_of_concentration', C0=0.0, Cs=1.0,
Cx=0.4795001221869535, D=1e-10, t=10000.0)
# -> x = 0.001 m, x_mm = 1, z = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diffusion depth: C0 = 0.2 wt%, Cs = 1.0 wt%, Cx = 0.6 wt%, D = 2e-11 m^2/s, t = 10 h	SOLVED	x = 0.000809388 m, x_mm = 0.809388, z = 0.476936

Activation energy and pre-exponential factor of diffusion — `diffusion_arrhenius_fit`

Q_d and D_0 of $D = D_0$ exp(-Q_d/RT) from diffusion coefficients at two or more temperatures, and D at another temperature.

$$\ln D = \ln D_0 - \frac{Q_d}{R}\frac{1}{T}$$

input	meaning	condition
`T`	temperatures in kelvin	
`D`	diffusion coefficients in m^2/s at those temperatures	

input	meaning	condition
`T_target`	temperature at which D is wanted (optional)	

output	meaning
`Q`	activation energy
`Q_kJmol`	activation energy in kJ/mol
`D0`	pre-exponential factor
`log10_D0`	log10 of D0 in m^2/s
`r_squared`	r^2 of the fit
`D_target`	D at T_target
`log10_D_target`	log10 of D at T_target

- **Method:** least-squares line of ln D against $1/T$: slope = -Q_d/R, intercept = ln D_0
- **Assumes:** equal lengths, at least two distinct temperatures; D rising with T
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('diffusion_arrhenius_fit', T=[773.0, 873.0], D=[5.8e-13, 5.3e-12])
# -> Q = 124136 J/mol, Q_kJmol = 124.136, D0 = 0.000141785 m^2/s, log10_D0 = -3.84837, r_squared =
1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Diffusion arrhenius fit: T = [773, 873], D = [5.8e-13, 5.3e-12]	SOLVED	Q = 124136 J/mol, Q_kJmol = 124.136, D0 = 0.000141785 m^2/s, log10_D0 = -3.84837, r_squared = 1
Diffusion arrhenius fit: T = [773, 873], D = [5.3e-12, 5.8e-13]	INVALID_INPUT	Activation energy and pre-exponential factor of diffusion: D must increase with temperature (a positive activation energ
Diffusion arrhenius fit: T = [773, 873, 973], D = [5.8e-13, 5.3e-12]	INVALID_INPUT	Activation energy and pre-exponential factor of diffusion: T and D must have the same length.

True stress and true strain from engineering values — `true_stress_and_strain`

True stress and true strain from the engineering stress and strain of a tensile test before necking (volume conserved).

$$\sigma_T = \sigma(1+\varepsilon), \quad \varepsilon_T = \ln(1+\varepsilon)$$

input	meaning	condition
`stress`	engineering stress	
`strain`	engineering strain (dimensionless)	

output	meaning
`true_stress`	true stress
`true_stress_MPa`	true stress in MPa
`true_strain`	true strain

- **Method:** $sigma_T = \sigma$ (1 + epsilon), epsilon_T = ln(1 + epsilon)
- **Assumes:** strain > -1; uniform deformation (before necking)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('true_stress_and_strain', stress=400000000.0, strain=0.2)
# -> true_stress = 4.8e+08 Pa, true_stress_MPa = 480, true_strain = 0.182322   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
True stress and strain: stress = 400 MPa, strain = 0.2	SOLVED	true_stress = 4.8e+08 Pa, true_stress_MPa = 480, true_strain = 0.182322
True stress and strain: stress = 400 MPa, strain = -2	INVALID_INPUT	True stress and true strain from engineering values: the engineering strain must exceed -1.

Elastic elongation and diameter change of a rod in tension — `elastic_deformation_of_a_rod`

Stress, axial strain, elongation, lateral strain and diameter change of a cylindrical rod under an axial force (Hooke's law and Poisson's ratio).

$$\varepsilon_z = \frac{F}{A_0 E}, \quad \varepsilon_x = -\nu \varepsilon_z$$

input	meaning	condition
`F`	axial force (tension positive)	
`d0`	original diameter	
`l0`	original length	
`E`	Young's modulus	
`nu`	Poisson's ratio	

output	meaning
`stress_MPa`	axial stress in MPa
`axial_strain`	axial strain
`elongation_mm`	elongation in mm
`lateral_strain`	lateral strain
`diameter_change_um`	change of diameter in micrometres (negative = contraction)

- **Method:** sigma = F/(pi d0^2/4), epsilon_z = sigma/E, epsilon_x = -nu epsilon_z, dl = epsilon_z l0, dd = epsilon_x d0
- **Assumes:** F, d0, l0, $E > 0$; $-1 < \nu < 0.5$; elastic, isotropic
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('elastic_deformation_of_a_rod', F=15000.0, d0=0.01, l0=0.3,
E=110000000000.0, nu=0.34)
# -> stress_MPa = 190.986, axial_strain = 0.00173624, elongation_mm = 0.520871, lateral_strain =
-0.00059032, diameter_change_um = -5.9032   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Elastic deformation: F = 15 kN, d0 = 10 mm, l0 = 300 mm, E = 110 GPa, nu = 0.34	SOLVED	stress_MPa = 190.986, axial_strain = 0.00173624, elongation_mm = 0.520871, lateral_strain = -0.00059032, diameter_change_um = -5.9032
Elastic deformation: F = 15 kN, d0 = 10 mm, l0 = 300 mm, E = 110 GPa, nu = 0.7	INVALID_INPUT	Elastic elongation and diameter change of a rod in tension: Poisson's ratio must lie between -1 and 0.5.

Shear modulus, bulk modulus and Lame constant from E and Poisson's ratio — `isotropic_elastic_constants`

The shear modulus G, bulk modulus K and Lame constant lambda of an isotropic material from Young's modulus E and Poisson's ratio nu.

$$G = \frac{E}{2(1+\nu)}, \quad K = \frac{E}{3(1-2\nu)}$$

input	meaning	condition
`E`	Young's modulus	
`nu`	Poisson's ratio	

output	meaning
`G`	shear modulus
`G_GPa`	shear modulus in GPa
`K`	bulk modulus
`K_GPa`	bulk modulus in GPa
`lame_GPa`	Lame's first constant in GPa

- **Method:** G = E/(2(1 + nu)), K = E/(3($1 - 2$ nu)), $\lambda = E$ nu/($(1 + \nu)$(1 - 2 nu))
- **Assumes:** $E > 0$; $-1 < \nu < 0.5$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('isotropic_elastic_constants', E=200000000000.0, nu=0.3)
# -> G = 7.69231e+10 Pa, G_GPa = 76.9231, K = 1.66667e+11 Pa, K_GPa = 166.667, lame_GPa = 115.385
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Elastic constants: E = 200 GPa, nu = 0.3	SOLVED	G = 7.69231e+10 Pa, G_GPa = 76.9231, K = 1.66667e+11 Pa, K_GPa = 166.667, lame_GPa = 115.385
Elastic constants: E = 200 GPa, nu = 0.5	INVALID_INPUT	Shear modulus, bulk modulus and Lame constant from E and Poisson's ratio: Poisson's ratio must lie between -1 and 0.5.

Young's modulus from elastic stress-strain data — `youngs_modulus_from_data`

Young's modulus as the slope of the least-squares line through stress-strain points of the elastic region.

$$E = \frac{\sum(\varepsilon_i - \bar{\varepsilon})(\sigma_i - \bar{\sigma})}{\sum(\varepsilon_i - \bar{\varepsilon})^2}$$

input	meaning	condition
`strain`	engineering strains (dimensionless)	
`stress`	engineering stresses in MPa	

output	meaning
`E_GPa`	Young's modulus in GPa
`E`	Young's modulus
`intercept_MPa`	intercept of the line in MPa
`r_squared`	r^2 of the fit

- **Method:** least-squares line stress = b0 + E strain
- **Assumes:** equal lengths, at least two distinct strains; stress in MPa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('youngs_modulus_from_data', strain=[0, 0.001, 0.002], stress=[0, 200, 400])
# -> E_GPa = 200, E = 2e+11 Pa, intercept_MPa = 0, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Young's modulus from data: strain = [0, 0.001, 0.002], stress = [0, 200, 400]	SOLVED	E_GPa = 200, E = 2e+11 Pa, intercept_MPa = 0, r_squared = 1
Young's modulus from data: strain = [0, 0.001, 0.002], stress = [0, 200]	INVALID_INPUT	Young's modulus from elastic stress-strain data: strain and stress must have the same length.

0.2 % offset yield strength from a stress-strain curve — `offset_yield_strength`

The yield strength where the stress-strain curve (piecewise linear through the data) meets a line of slope E offset by 0.2 % strain (or another offset).

$$\sigma(\varepsilon_y) = E(\varepsilon_y - 0.002)$$

input	meaning	condition
`strain`	engineering strains, increasing	
`stress`	engineering stresses in MPa	
`E`	Young's modulus	
`offset`	strain offset (default 0.002)	

output	meaning
`yield_MPa`	offset yield strength in MPa
`yield_strain`	strain at the intersection

- **Method:** first crossing of the data polyline with $\sigma = E$ (epsilon - offset), solved on that segment
- **Assumes:** strains strictly increasing; the curve starts above the offset line and crosses it
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('offset_yield_strength', strain=[0, 0.002, 0.004, 0.006, 0.008],
stress=[0, 400, 520, 560, 580], E=200000000000.0)
# -> yield_MPa = 533.333, yield_strain = 0.00466667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Offset yield strength: strain = [0, 0.002, 0.004, 0.006, 0.008], stress = [0, 400, 520, 560, 580], E = 200 GPa	SOLVED	yield_MPa = 533.333, yield_strain = 0.00466667
Offset yield strength: strain = [0, 0.001, 0.002], stress = [0, 200, 400], E = 200 GPa	INVALID_INPUT	0.2 % offset yield strength from a stress-strain curve: the curve never crosses the offset line: the data end before yie

Toughness (area under the stress-strain curve) — `tensile_toughness`

The energy absorbed per unit volume up to fracture (trapezoidal area under the stress-strain data), the tensile strength and the fracture strain.

$$U_T = \int_0^{\varepsilon_f} \sigma \, d\varepsilon$$

input	meaning	condition
`strain`	engineering strains, non-decreasing	
`stress`	engineering stresses in MPa	

output	meaning
`toughness_MJ_m3`	toughness in MJ/m^3
`toughness`	toughness
`uts_MPa`	tensile strength (largest stress) in MPa
`fracture_strain`	strain at the last point

- **Method:** trapezoidal rule over the data points
- **Assumes:** equal lengths; strains non-decreasing; stress in MPa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('tensile_toughness', strain=[0, 0.002, 0.05, 0.1, 0.15], stress=[0, 400, 500, 520, 480])
# -> toughness_MJ_m3 = 72.5, toughness = 7.25e+07 J/m^3, uts_MPa = 520, fracture_strain = 0.15  (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Tensile toughness: strain = [0, 0.002, 0.05, 0.1, 0.15], stress = [0, 400, 500, 520, 480]	SOLVED	toughness_MJ_m3 = 72.5, toughness = 7.25e+07 J/m^3, uts_MPa = 520, fracture_strain = 0.15
Tensile toughness: strain = [0, 0.1, 0.05], stress = [0, 400, 500]	INVALID_INPUT	Toughness (area under the stress-strain curve): strains must not decrease.

Modulus of resilience — `modulus_of_resilience`

The elastic strain energy per unit volume stored up to yielding: $U_r = sigma_y^2/(2E)$.

$$U_r = \frac{\sigma_y^2}{2E}$$

input	meaning	condition
`sy`	yield strength	
`E`	Young's modulus	

output	meaning
`Ur`	modulus of resilience
`Ur_MJ_m3`	modulus of resilience in MJ/m^3 (= MPa)
`yield_strain`	strain at yield

- **Method:** area under the linear elastic part of the curve
- **Assumes:** sy, $E > 0$; linear elastic up to yield
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('modulus_of_resilience', sy=415000000.0, E=207000000000.0)
# -> Ur = 416002 J/m^3, Ur_MJ_m3 = 0.416002, yield_strain = 0.00200483   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Modulus of resilience: sy = 415 MPa, E = 207 GPa	SOLVED	Ur = 416002 J/m^3, Ur_MJ_m3 = 0.416002, yield_strain = 0.00200483

question	outcome	result
Modulus of resilience: sy = 415 MPa	NEEDS_INPUT	modulus_of_resilience needs E (Young's modulus). Example: solve('modulus_of_resilience', sy='4.15e+08 Pa', E='2.07e+11 P

Strain-hardening exponent and strength coefficient (Hollomon) — `strain_hardening_exponent`

The strain-hardening exponent n and strength coefficient K of $sigma_T = K$ epsilon_T^n from true stress-strain points of the plastic region, and the stress at another strain.

$$\ln \sigma_T = \ln K + n \ln \varepsilon_T$$

input	meaning	condition
`true_strain`	true plastic strains	
`true_stress`	true stresses in MPa	
`strain_target`	true strain at which the stress is wanted (optional)	

output	meaning
`n`	strain-hardening exponent
`K_MPa`	strength coefficient in MPa
`r_squared`	r^2 of the log-log fit
`stress_target_MPa`	true stress at the target strain in MPa

- **Method:** least-squares line of ln sigma_T against ln epsilon_T
- **Assumes:** equal lengths, at least two distinct strains, all values > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('strain_hardening_exponent', true_strain=[0.1, 0.2], true_stress=
[415, 470])
# -> n = 0.179549, K_MPa = 627.477, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Strain hardening exponent: true_strain = [0.1, 0.2], true_stress = [415, 470]	SOLVED	n = 0.179549, K_MPa = 627.477, r_squared = 1
Strain hardening exponent: true_strain = [0.1, -0.2], true_stress = [415, 470]	INVALID_INPUT	Strain-hardening exponent and strength coefficient (Hollomon): every true strain and stress (plastic region) must be a p

Hall-Petch grain-size strengthening — `hall_petch`

sigma_0 and k of $sigma_y = sigma_0 + k\, d^{-1/2}$ from yield strengths at two or more grain sizes, and the yield strength at another grain size.

$$\sigma_y = \sigma_0 + k_y d^{-1/2}$$

input	meaning	condition
`d_um`	grain diameters in micrometres	
`yield_MPa`	yield strengths in MPa	
`d_target_um`	grain diameter in micrometres at which the yield strength is wanted (optional)	

output	meaning
`sigma0_MPa`	friction stress sigma_0 in MPa
`k_MPa_sqrt_um`	k in MPa $um^{0.5}$
`k_MPa_sqrt_m`	k in MPa $m^{0.5}$
`r_squared`	r^2 of the fit
`yield_target_MPa`	yield strength at the target grain size in MPa

- **Method:** least-squares line of sigma_y against $d^{-1/2}$
- **Assumes:** equal lengths, at least two distinct grain sizes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('hall_petch', d_um=[100, 4], yield_MPa=[150, 350])
# -> sigma0_MPa = 100, k_MPa_sqrt_um = 500, k_MPa_sqrt_m = 0.5, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hall-Petch: d_um = [100, 4], yield_MPa = [150, 350], d_target_um = 25	SOLVED	sigma0_MPa = 100, k_MPa_sqrt_um = 500, k_MPa_sqrt_m = 0.5, r_squared = 1, yield_target_MPa = 200
Hall-Petch: d_um = [100, 100], yield_MPa = [150, 350]	INVALID_INPUT	Hall-Petch grain-size strengthening: the x values must not all be equal.

Thermal stress in a constrained body — `thermal_stress`

The stress from a temperature change in a body held in one direction (rod between walls), two (thin film or plate) or three (fully constrained).

$$\sigma = -E\alpha\Delta T, \quad -\frac{E\alpha\Delta T}{1-\nu}, \quad -\frac{E\alpha\Delta T}{1-2\nu}$$

input	meaning	condition
`E`	Young's modulus	
`alpha`	linear thermal expansion coefficient	
`dT`	temperature change (positive = heating)	
`constraint`	constraint (uniaxial, biaxial, triaxial; default uniaxial)	
`nu`	Poisson's ratio (needed for biaxial and triaxial)	

output	meaning
`stress`	thermal stress (negative = compression)
`stress_MPa`	thermal stress in MPa

- **Method:** $\sigma = -E$ alpha dT (uniaxial), / $(1 - \nu)$ (biaxial), / (1 - 2 nu) (triaxial)
- **Assumes:** E, $\alpha > 0$; $-1 < \nu < 0.5$ when needed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('thermal_stress', E=97000000000.0, alpha=2e-05, dT=60.0)
# -> stress = -1.164e+08 Pa, stress_MPa = -116.4   (SOLVED)
```

```
materials_science_ai999x.solve('thermal_stress', E=70000000000.0, alpha=2.3e-05, dT=-100.0,
constraint='biaxial', nu=0.33)
# -> stress = 2.40299e+08 Pa, stress_MPa = 240.299   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thermal stress: E = 97 GPa, alpha = 20e-6 1/K, dT = 60 K	SOLVED	stress = -1.164e+08 Pa, stress_MPa = -116.4
Thermal stress: E = 70 GPa, alpha = 23e-6 1/K, dT = -100 K, constraint = biaxial	INVALID_INPUT	Thermal stress in a constrained body: Poisson's ratio is needed for a biaxial or triaxial constraint.

Elastic moduli of a continuous-fibre composite (rule of mixtures) —

`composite_rule_of_mixtures`

The longitudinal (isostrain) and transverse (isostress) moduli of an aligned continuous-fibre composite and the fraction of the load carried by the fibres.

$$E_{cl} = E_m V_m + E_f V_f, \quad \frac{1}{E_{ct}} = \frac{V_m}{E_m} + \frac{V_f}{E_f}$$

input	meaning	condition
`Ef`	fibre modulus	
`Em`	matrix modulus	
`Vf`	fibre volume fraction (0 to 1)	

output	meaning
`E_long_GPa`	longitudinal modulus in GPa
`E_trans_GPa`	transverse modulus in GPa
`fiber_load_fraction`	fraction of a longitudinal load carried by the fibres
`E_long`	longitudinal modulus
`E_trans`	transverse modulus

- **Method:** $E_l = Ef$ Vf + Em Vm (Voigt), $1/E_t = Vf/Ef + Vm/Em$ (Reuss), $Ff/F = Ef$ Vf / E_l
- **Assumes:** moduli > 0; $0 < Vf < 1$; perfect bonding
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('composite_rule_of_mixtures', Ef=69000000000.0, Em=3400000000.0,
Vf=0.4)
# -> E_long_GPa = 29.64, E_trans_GPa = 5.48644, fiber_load_fraction = 0.931174, E_long = 2.964e+10
Pa, E_trans = 5.48644e+09 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rule of mixtures: Ef = 69 GPa, Em = 3.4 GPa, Vf = 0.4	SOLVED	E_long_GPa = 29.64, E_trans_GPa = 5.48644, fiber_load_fraction = 0.931174, E_long = 2.964e+10 Pa, E_trans = 5.48644e+09 Pa
Rule of mixtures: Ef = 69 GPa, Em = 3.4 GPa, Vf = 1.4	INVALID_INPUT	Elastic moduli of a continuous-fibre composite (rule of mixtures): the fibre volume fraction must lie between 0 and 1.

Critical fibre length — `critical_fiber_length`

The critical length $l_c = sigma_f$ d / (2 tau_c) a fibre needs to be loaded to its strength through the fibre-matrix interface.

$$l_c = \frac{\sigma_f^* d}{2\tau_c}$$

input	meaning	condition
`sigma_f`	fibre tensile strength	
`d`	fibre diameter	
`tau_c`	fibre-matrix bond strength (or matrix shear yield strength)	

output	meaning
`lc`	critical fibre length
`lc_mm`	critical length in mm
`aspect_ratio`	l_c/d

- **Method:** $l_c = sigma_f$ d / (2 tau_c)
- **Assumes:** all inputs > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('critical_fiber_length', sigma_f=3500000000.0, d=1e-05,
tau_c=20000000.0)
# -> lc = 0.000875 m, lc_mm = 0.875, aspect_ratio = 87.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical fiber length: sigma_f = 3.5 GPa, d = 10 um, tau_c = 20 MPa	SOLVED	lc = 0.000875 m, lc_mm = 0.875, aspect_ratio = 87.5
Critical fibre length: sigma_f = 3.5 GPa, d = 10 um	NEEDS_INPUT	critical_fiber_length needs tau_c (fibre-matrix bond strength (or matrix shear yield strength)). Example: solve('critica

Griffith critical stress of a brittle solid — `griffith_critical_stress`

The stress at which a crack of length 2a (internal) or a (surface) propagates in a brittle solid: $sigma_c = sqrt(2 E gamma_s / (pi a))$.

$$\sigma_c = \sqrt{\frac{2E\gamma_s}{\pi a}}$$

input	meaning	condition
`E`	Young's modulus	
`gamma_s`	specific surface energy	
`a`	half-length of an internal crack (or length of a surface crack)	

output	meaning
`sigma_c`	critical stress
`sigma_c_MPa`	critical stress in MPa

- **Method:** Griffith energy balance
- **Assumes:** all inputs > 0; ideally brittle (no plastic work), plane stress
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('griffith_critical_stress', E=69000000000.0, gamma_s=0.3, a=5e-07)
# -> sigma_c = 1.62345e+08 Pa, sigma_c_MPa = 162.345   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Griffith critical stress: E = 69 GPa, gamma_s = 0.3 J/m^2, a = 0.5 um	SOLVED	sigma_c = 1.62345e+08 Pa, sigma_c_MPa = 162.345
Griffith critical stress: E = 69 GPa, gamma_s = 0.3 J/m^2	NEEDS_INPUT	griffith_critical_stress needs a (half-length of an internal crack (or length of a surface crack)). Example: solve('grif

Fracture stress from the fracture toughness — `fracture_critical_stress`

The stress at which a crack of length a fails in a material of plane-strain fracture toughness K_IC: sigma_c = K_IC / (Y sqrt(pi a)).

$$\sigma_c = \frac{K_{Ic}}{Y\sqrt{\pi a}}$$

input	meaning	condition
`K_IC`	plane-strain fracture toughness	
`a`	crack length (surface) or half-length (internal)	
`Y`	geometry factor Y (default 1)	

output	meaning
`sigma_c`	critical stress
`sigma_c_MPa`	critical stress in MPa

- **Method:** $K_I = Y$ sigma sqrt(pi a) set equal to K_IC
- **Assumes:** all inputs > 0; linear-elastic fracture mechanics
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('fracture_critical_stress', K_IC=26000000.0, a=0.002, Y=1.2)
# -> sigma_c = 2.73339e+08 Pa, sigma_c_MPa = 273.339   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fracture stress: K_IC = 26 MPa*m^0.5, a = 2 mm, Y = 1.2	SOLVED	sigma_c = 2.73339e+08 Pa, sigma_c_MPa = 273.339
Fracture stress: K_IC = 26 MPa*m^0.5	NEEDS_INPUT	fracture_critical_stress needs a (crack length (surface) or half-length (internal)). Example: solve('fracture_critical_s

Critical crack length from the fracture toughness — `critical_crack_length`

The largest crack a part can carry at stress sigma without fracture: a_c = (K_IC / (Y sigma))^$2/pi$.

$$a_c = \frac{1}{\pi}\left(\frac{K_{Ic}}{Y\sigma}\right)^2$$

input	meaning	condition
`K_IC`	plane-strain fracture toughness	
`sigma`	applied stress	
`Y`	geometry factor Y (default 1)	

output	meaning
`a_c`	critical crack length
`a_c_mm`	critical crack length in mm

- **Method:** $K_I = Y$ sigma sqrt(pi a) set equal to K_IC
- **Assumes:** all inputs > 0; linear-elastic fracture mechanics
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('critical_crack_length', K_IC=55000000.0, sigma=1000000000.0, Y=1.0)
# -> a_c = 0.000962887 m, a_c_mm = 0.962887   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical crack length: K_IC = 55 MPa*m^0.5, sigma = 1000 MPa	SOLVED	a_c = 0.000962887 m, a_c_mm = 0.962887
Critical crack length: K_IC = 55 MPa*m^0.5, sigma = -10 MPa	INVALID_INPUT	Critical crack length from the fracture toughness: K_IC, the stress and Y must be a positive number.

Weibull probability of failure of a brittle part — `weibull_failure_probability`

The probability that a brittle (ceramic) part fails at stress sigma, from the Weibull modulus m and characteristic strength sigma_0, with an optional volume ratio V/V_0.

$$P_f = 1 - \exp\left[-\frac{V}{V_0}\left(\frac{\sigma}{\sigma_0}\right)^m\right]$$

input	meaning	condition
`sigma`	applied stress	
`sigma0`	characteristic strength (63.2 % failure at V0)	
`m`	Weibull modulus	
`V_ratio`	stressed volume over the reference volume V/V_0 (default 1)	

output	meaning
`P_failure`	probability of failure
`P_survival`	probability of survival

- **Method:** $P_f = 1 - exp$(-(V/V_0)(sigma/sigma0)^m)
- **Assumes:** all inputs > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('weibull_failure_probability', sigma=300000000.0,
sigma0=400000000.0, m=10.0)
# -> P_failure = 0.0547573, P_survival = 0.945243   (SOLVED)
```

```
materials_science_ai999x.solve('weibull_failure_probability', sigma=400000000.0,
sigma0=400000000.0, m=5.0, V_ratio=2.0)
# -> P_failure = 0.864665, P_survival = 0.135335   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weibull failure probability: sigma = 300 MPa, sigma0 = 400 MPa, m = 10	SOLVED	P_failure = 0.0547573, P_survival = 0.945243
Weibull failure probability: sigma = 300 MPa, sigma0 = 400 MPa, m = -2	INVALID_INPUT	Weibull probability of failure of a brittle part: the stresses, the Weibull modulus and the volume ratio must be a posit

Design stress for a target failure probability (Weibull) — `weibull_design_stress`

The stress at which a brittle part reaches a given failure probability, from the Weibull modulus and characteristic strength (and V/V0).

$$\sigma = \sigma_0 \left[-\frac{V_0}{V} \ln(1 - P_f) \right]^{1/m}$$

input	meaning	condition
`P_failure`	allowed probability of failure (0 to 1)	
`sigma0`	characteristic strength	
`m`	Weibull modulus	
`V_ratio`	V/V_0 (default 1)	

output	meaning
`sigma`	design stress
`sigma_MPa`	design stress in MPa

- **Method:** $\sigma = sigma_0$ (-ln(1 - P_f) V0/V)^(1/m)
- **Assumes:** 0 < P_failure < 1; sigma0, m, $V/V_0 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('weibull_design_stress', P_failure=0.001, sigma0=400000000.0,
m=10.0)
# -> sigma = 2.00485e+08 Pa, sigma_MPa = 200.485   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weibull design stress: P_failure = 0.001, sigma0 = 400 MPa, m = 10	SOLVED	sigma = 2.00485e+08 Pa, sigma_MPa = 200.485
Weibull design stress: P_failure = 1.5, sigma0 = 400 MPa, m = 10	INVALID_INPUT	Design stress for a target failure probability (Weibull): the failure probability must lie between 0 and 1.

Weibull modulus from a set of measured strengths — `weibull_modulus_fit`

The Weibull modulus m and characteristic strength sigma_0 from measured fracture strengths, by median-rank regression (Bernard's ranks).

$$\ln \ln \frac{1}{1 - F_i} = m \ln \sigma_i - m \ln \sigma_0$$

input	meaning	condition
`strengths`	fracture strengths (any one unit, e.g. MPa)	

output	meaning
`m`	Weibull modulus
`sigma0`	characteristic strength (unit of the data)
`r_squared`	r^2 of the fit

- **Method:** sort; $F_i = (i - 0.3)/(n + 0.4)$; least-squares line of ln(-ln(1 - F_i)) against ln(sigma_i): slope m, intercept -m ln(sigma_0)
- **Assumes:** at least three strengths, not all equal, all > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('weibull_modulus_fit', strengths=[310, 342, 365, 380, 395, 410,
432])
# -> m = 9.55748, sigma0 = 394.709, r_squared = 0.999566   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weibull modulus: strengths = [310, 342, 365, 380, 395, 410, 432]	SOLVED	m = 9.55748, sigma0 = 394.709, r_squared = 0.999566
Weibull modulus: strengths = [310, 342]	INVALID_INPUT	strengths: needs at least 3 entries, got 2.

Stress exponent and activation energy of steady-state creep — `creep_parameters_fit`

n and Q_c of the power-law creep model rate = A σ^n exp(-Q_c/RT) from steady-state creep rates at several stresses and temperatures (least squares).

$$\dot{\varepsilon}_s = A\sigma^n e^{-Q_c/RT}$$

input	meaning	condition
`stress_MPa`	stresses in MPa	
`T`	temperatures in kelvin	
`rate`	steady-state creep rates in $1/s$	

output	meaning
`n`	stress exponent
`Q_kJmol`	creep activation energy in kJ/mol

- **Method:** least squares of ln(rate) = ln A + n ln(sigma) - Q_c/(R T)
- **Assumes:** equal lengths (at least 3); stress and temperature both vary independently; all values > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('creep_parameters_fit', stress_MPa=[100, 200, 100], T=[900, 900,
1000], rate=[1e-07, 1.6e-06, 2.5e-06])
# -> n = 4, Q_kJmol = 240.869   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Creep parameters: stress_MPa = [100, 200, 100], T = [900, 900, 1000], rate = [1e-7, 1.6e-6, 2.5e-6]	SOLVED	n = 4, Q_kJmol = 240.869
Creep parameters: stress_MPa = [100, 200, 300], T = [900, 900, 900], rate = [1e-7, 1.6e-6, 2.5e-6]	INVALID_INPUT	Stress exponent and activation energy of steady-state creep: the stresses and temperatures must both vary, and not in pr

Stress relaxation of a Maxwell viscoelastic solid — `maxwell_stress_relaxation`

The stress remaining after time t in a polymer held at constant strain, modelled as a Maxwell element: sigma(t) = sigma_0 exp(-t E/eta).

$$\sigma(t) = \sigma_0 e^{-t/\tau}, \quad \tau = \frac{\eta}{E}$$

input	meaning	condition
`sigma0`	initial stress	
`E`	spring modulus	
`eta`	dashpot viscosity	
`t`	time held	

output	meaning
`sigma_t`	stress at time t
`sigma_t_MPa`	stress at time t in MPa
`tau`	relaxation time eta/E
`ratio`	sigma(t)/sigma_0

- **Method:** sigma(t) = sigma_0 exp(-t/tau), $\tau = \eta/E$
- **Assumes:** sigma0, E, $\eta > 0$; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('maxwell_stress_relaxation', sigma0=10000000.0, E=1000000000.0,
eta=1000000000000.0, t=1000.0)
# -> sigma_t = 3.67879e+06 Pa, sigma_t_MPa = 3.67879, tau = 1000 s, ratio = 0.367879   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maxwell stress relaxation: sigma0 = 10 MPa, E = 1 GPa, eta = 1e12 Pa*s, t = 1000 s	SOLVED	sigma_t = 3.67879e+06 Pa, sigma_t_MPa = 3.67879, tau = 1000 s, ratio = 0.367879
Maxwell stress relaxation: sigma0 = 10 MPa, E = 1 GPa, eta = 1e12 Pa*s	NEEDS_INPUT	maxwell_stress_relaxation needs t (time held). Example: solve('maxwell_stress_relaxation', sigma0='1e+07 Pa', E='1e+09 P

Creep strain of a Kelvin-Voigt viscoelastic solid — `kelvin_voigt_creep`

The strain after time t under constant stress for a Kelvin-Voigt (spring and dashpot in parallel) material: eps(t) = (sigma_0/E)(1 - exp(-t E/eta)).

$$\varepsilon(t) = \frac{\sigma_0}{E}\left(1 - e^{-t/\tau}\right)$$

input	meaning	condition
`sigma0`	applied stress	
`E`	spring modulus	
`eta`	dashpot viscosity	
`t`	time under load	

output	meaning
`strain`	creep strain at time t
`fraction_of_elastic`	strain over the final strain $sigma_0/E$
`tau`	retardation time eta/E

- **Method:** eps(t) = (sigma_0/E)(1 - exp(-t/tau)), $\tau = \eta/E$
- **Assumes:** sigma0, E, $\eta > 0$; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('kelvin_voigt_creep', sigma0=5000000.0, E=500000000.0,
eta=100000000000.0, t=200.0)
# -> strain = 0.00632121, fraction_of_elastic = 0.632121, tau = 200 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kelvin-Voigt creep: sigma0 = 5 MPa, E = 500 MPa, eta = 1e11 Pa*s, t = 200 s	SOLVED	strain = 0.00632121, fraction_of_elastic = 0.632121, tau = 200 s

Resolved shear stress and Schmid factor of a slip system — `schmid_factor`

The Schmid factor cos(phi) cos(lambda) of a slip system (plane normal and slip direction) for a tensile load along a direction of a cubic crystal, the resolved shear stress and the stress to start slip.

$$\tau_R = \sigma \cos\phi \cos\lambda, \quad \sigma_y = \frac{\tau_{\mathrm{crss}}}{\cos\phi \cos\lambda}$$

input	meaning	condition
`load`	loading direction [u v w]	
`plane`	slip-plane normal (h k l)	
`direction`	slip direction [u v w]	
`sigma`	applied tensile stress (optional)	

input	meaning	condition
`tau_crss`	critical resolved shear stress (optional)	

output	meaning
`schmid_factor`	|cos(phi) cos(lambda)|
`phi_deg`	angle between load and plane normal
`lambda_deg`	angle between load and slip direction
`tau_R_MPa`	resolved shear stress in MPa
`yield_MPa`	tensile stress at which slip starts, in MPa

- **Method:** $tau_R = \sigma$ cos(phi) cos(lambda); sigma_y = tau_CRSS / (cos(phi) cos(lambda))
- **Assumes:** cubic crystal (indices as Cartesian directions); slip direction in the slip plane
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('schmid_factor', load=[0, 1, 0], plane=[1, 1, 0], direction=[-1, 1,
1], sigma=52000000.0)
# -> schmid_factor = 0.408248, phi_deg = 45, lambda_deg = 54.7356, tau_R_MPa = 21.2289   (SOLVED)
```

```
materials_science_ai999x.solve('schmid_factor', load=[0, 0, 1], plane=[1, 1, 1], direction=[1, -1,
0])
# -> schmid_factor = 0, phi_deg = 54.7356, lambda_deg = 90   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Schmid factor: load = [0, 1, 0], plane = [1, 1, 0], direction = [-1, 1, 1], sigma = 52 MPa	SOLVED	schmid_factor = 0.408248, phi_deg = 45, lambda_deg = 54.7356, tau_R_MPa = 21.2289
Schmid factor: load = [0, 1, 0], plane = [1, 1, 0], direction = [1, 1, 1]	INVALID_INPUT	Resolved shear stress and Schmid factor of a slip system: the slip direction must lie in the slip plane (h u + k v + l w
Schmid factor: load = [0, 0, 1], plane = [1, 1, 1], direction = [1, -1, 0], tau_crss = 1 MPa	INVALID_INPUT	Resolved shear stress and Schmid factor of a slip system: the Schmid factor is 0: this system never yields under this lo

Most favourable slip system of an FCC or BCC crystal — `max_schmid_factor`

The largest Schmid factor over the 12 {111}<110> (FCC) or {110}<111> (BCC) slip systems for a tensile load direction, how many systems share it, and the yield stress from the CRSS.

$$m = \max_{\text{systems}} \left| \cos\phi \cos\lambda \right|$$

input	meaning	condition
`structure`	crystal structure (FCC, BCC)	
`load`	loading direction [u v w]	
`tau_crss`	critical resolved shear stress (optional)	

output	meaning
`max_schmid_factor`	largest Schmid factor
`active_systems`	number of systems with that factor
`systems`	slip systems considered
`yield_MPa`	tensile yield stress in MPa

- **Method:** Schmid factor of each of the 12 systems; the maximum (ties within $1e-9$ counted)
- **Assumes:** nonzero loading direction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('max_schmid_factor', structure='FCC', load=[0, 0, 1])
# -> max_schmid_factor = 0.408248, active_systems = 8, systems = 12   (SOLVED)
```

```
materials_science_ai999x.solve('max_schmid_factor', structure='FCC', load=[1, 1, 1])
# -> max_schmid_factor = 0.272166, active_systems = 6, systems = 12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum Schmid factor: structure = FCC, load = [0, 0, 1]	SOLVED	max_schmid_factor = 0.408248, active_systems = 8, systems = 12
Maximum Schmid factor: structure = FCC, load = [0, 0, 0]	INVALID_INPUT	Most favourable slip system of an FCC or BCC crystal: the loading direction must be a nonzero 3-vector.

Phase fractions by the lever rule — `lever_rule`

The mass fractions (and masses) of the two phases of an alloy of overall composition C0 in a two-phase region whose tie line ends at C1 and C2.

$$W_1 = \frac{C_2 - C_0}{C_2 - C_1}, \quad W_2 = \frac{C_0 - C_1}{C_2 - C_1}$$

input	meaning	condition
`C0`	overall composition (wt%)	
`C1`	composition of phase 1 (wt%)	
`C2`	composition of phase 2 (wt%)	
`mass`	total mass of the alloy (optional)	

output	meaning
`W1`	mass fraction of phase 1
`W2`	mass fraction of phase 2
`m1`	mass of phase 1
`m2`	mass of phase 2

- **Method:** $W_1 = (C_2 - C_0)/(C_2 - C_1)$, $W_2 = (C_0 - C_1)/(C_2 - C_1)$

- **Assumes:** $C_1 \neq C_2$; C0 between C1 and C2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('lever_rule', C0=35.0, C1=31.5, C2=42.5)
# -> W1 = 0.681818, W2 = 0.318182   (SOLVED)
```

```
materials_science_ai999x.solve('lever_rule', C0=40.0, C1=20.0, C2=60.0, mass=3.0)
# -> W1 = 0.5, W2 = 0.5, m1 = 1.5 kg, m2 = 1.5 kg   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lever rule: C0 = 35 wt%, C1 = 31.5 wt%, C2 = 42.5 wt%	SOLVED	W1 = 0.681818, W2 = 0.318182
Lever rule: C0 = 50 wt%, C1 = 31.5 wt%, C2 = 42.5 wt%	INVALID_INPUT	Phase fractions by the lever rule: the overall composition C0 must lie between the two phase compositions.

Primary and eutectic microconstituent fractions — `eutectic_microconstituents`

The fractions of primary phase and eutectic microconstituent, and of total alpha and beta, in a binary eutectic alloy cooled just below the eutectic temperature.

$$W_{\alpha'} = \frac{C_E - C_0}{C_E - C_\alpha}, \quad W_e = \frac{C_0 - C_\alpha}{C_E - C_\alpha}$$

input	meaning	condition
`C0`	alloy composition (wt%)	
`C_alpha`	composition of alpha at the eutectic temperature	
`C_E`	eutectic composition	
`C_beta`	composition of beta at the eutectic temperature	

output	meaning
`W_primary`	fraction of primary (proeutectic) phase
`W_eutectic`	fraction of eutectic microconstituent
`W_alpha_total`	total fraction of alpha
`W_beta_total`	total fraction of beta
`primary_phase`	the primary phase (alpha for hypoeutectic, beta for hypereutectic)

- **Method:** lever rule on the tie line just above the eutectic (primary phase vs liquid of C_E) and just below it (alpha vs beta)
- **Assumes:** $C_alpha < C_E < C_beta$; $C_alpha < C_0 < C_beta$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('eutectic_microconstituents', C0=25.0, C_alpha=18.3, C_E=61.9,
C_beta=97.8)
# -> W_primary = 0.84633, W_eutectic = 0.15367, W_alpha_total = 0.915723, W_beta_total = 0.0842767,
primary_phase = alpha   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Eutectic microconstituents: C0 = 25 wt%, C_alpha = 18.3 wt%, C_E = 61.9 wt%, C_beta = 97.8 wt%	SOLVED	W_primary = 0.84633, W_eutectic = 0.15367, W_alpha_total = 0.915723, W_beta_total = 0.0842767, primary_phase = alpha
Eutectic microconstituents: C0 = 10 wt%, C_alpha = 18.3 wt%, C_E = 61.9 wt%, C_beta = 97.8 wt%	INVALID_INPUT	Primary and eutectic microconstituent fractions: C0 must lie between the two phase compositions.

Proeutectoid phase and pearlite in a plain-carbon steel — `iron_carbon_microconstituents`

The fractions of proeutectoid ferrite or cementite and of pearlite, and of total ferrite and cementite, in a steel cooled slowly just below the eutectoid temperature.

$$W_p = \frac{C_0 - 0.022}{0.76 - 0.022}\ (\text{hypo}), \quad W_p = \frac{6.70 - C_0}{6.70 - 0.76}\ (\text{hyper})$$

input	meaning	condition
`C0`	carbon content in wt% (0.022 to 2.14)	

output	meaning
`W_proeutectoid`	fraction of proeutectoid ferrite or cementite
`W_pearlite`	fraction of pearlite
`W_ferrite_total`	total fraction of ferrite
`W_cementite_total`	total fraction of cementite
`proeutectoid_phase`	ferrite (hypoeutectoid) or cementite (hypereutectoid)

- **Method:** lever rule with the eutectoid at 0.76 wt% C, ferrite 0.022 wt% C and cementite 6.70 wt% C
- **Assumes:** $0.022 < C_0 < 2.14$ wt% C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('iron_carbon_microconstituents', C0=0.35)
# -> W_proeutectoid = 0.555556, W_pearlite = 0.444444, W_ferrite_total = 0.950883,
W_cementite_total = 0.0491165, proeutectoid_phase = ferrite   (SOLVED)
```

```
materials_science_ai999x.solve('iron_carbon_microconstituents', C0=1.2)
# -> W_proeutectoid = 0.0740741, W_pearlite = 0.925926, W_ferrite_total = 0.8236, W_cementite_total
= 0.1764, proeutectoid_phase = cementite   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Iron-carbon microconstituents: C0 = 0.35 wt%	SOLVED	W_proeutectoid = 0.555556, W_pearlite = 0.444444, W_ferrite_total = 0.950883, W_cementite_total = 0.0491165, proeutectoid_phase = ferrite
Iron-carbon microconstituents: C0 = 3 wt%	INVALID_INPUT	Proeutectoid phase and pearlite in a plain-carbon steel: a steel: the carbon content must lie between 0.022 and 2.14 wt%

Degrees of freedom by Gibbs' phase rule — `gibbs_phase_rule`

The number of degrees of freedom $F = C - P + 2$ (or + 1 at fixed pressure) of a system of C components and P phases in equilibrium.

$$F = C - P + 2$$

input	meaning	condition
`C`	number of components	
`P`	number of phases	
`pressure`	pressure variable or held constant (default variable)	

output	meaning
`F`	degrees of freedom

- **Method:** $F = C - P + 2$ (variable pressure) or $C - P + 1$ (constant pressure, condensed systems)
- **Assumes:** $F \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('gibbs_phase_rule', C=1, P=3)
# -> F = 0   (SOLVED)
```

```
materials_science_ai999x.solve('gibbs_phase_rule', C=2, P=2, pressure='constant')
# -> F = 1   (SOLVED)
```

```
materials_science_ai999x.solve('gibbs_phase_rule', C=2, P=3, pressure='constant')
# -> F = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gibbs phase rule: C = 2, P = 3, pressure = constant	SOLVED	F = 0
Gibbs phase rule: C = 1, P = 4	INVALID_INPUT	Degrees of freedom by Gibbs' phase rule: more phases than the phase rule allows in equilibrium.

Critical nucleus radius and activation energy (homogeneous nucleation) — `homogeneous_nucleation`

The critical radius r* and the energy barrier dG* for homogeneous nucleation of a solid from its undercooled melt.

$$r^* = -\frac{2\gamma}{\Delta G_v}, \quad \Delta G^* = \frac{16\pi\gamma^3}{3\Delta G_v^2}, \quad \Delta G_v = -\frac{\Delta H_f \, \Delta T}{T_m}$$

input	meaning	condition
`gamma`	solid-liquid interfacial energy	
`dHf`	latent heat of fusion per volume	
`Tm`	melting temperature	
`T`	temperature of the undercooled liquid	

output	meaning
`undercooling`	undercooling Tm - T
`dGv_MJ_m3`	volume free-energy change in MJ/m^3
`r_star`	critical radius
`r_star_nm`	critical radius in nm
`dG_star`	activation free energy
`dG_star_eV`	activation free energy in eV

- **Method:** dGv = -dHf (Tm - T)/Tm; r* = -2 gamma/dGv; dG* = 16 pi gamma^3/(3 dGv^2)
- **Assumes:** gamma, dHf, $Tm > 0$; $0 < T < Tm$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('homogeneous_nucleation', gamma=0.132, dHf=1160000000.0, Tm=1337.15,
T=1107.15)
# -> undercooling = 230 K, dGv_MJ_m3 = -199.529, r_star = 1.32312e-09 m, r_star_nm = 1.32312,
dG_star = 9.67964e-19 J, dG_star_eV = 6.04155   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Homogeneous nucleation: gamma = 0.132 J/m^2, dHf = 1.16e9 J/m^3, Tm = 1064 degC, T = 834 degC	SOLVED	undercooling = 230 K, dGv_MJ_m3 = -199.529, r_star = 1.32312e-09 m, r_star_nm = 1.32312, dG_star = 9.67964e-19 J, dG_star_eV = 6.04155
Homogeneous nucleation: gamma = 0.132 J/m^2, dHf = 1.16e9 J/m^3, Tm = 1064 degC, T = 1100 degC	INVALID_INPUT	Critical nucleus radius and activation energy (homogeneous nucleation): the temperature must lie below the melting point

Avrami (JMAK) transformation kinetics — `avrami_kinetics`

The Avrami exponent n and rate constant k of $y = 1 - exp$(-k t^n) from transformed fractions at two or more times, the half-transformation time and the time for a target fraction.

$$y = 1 - e^{-kt^n}, \quad \ln\ln\frac{1}{1-y} = \ln k + n\ln t$$

input	meaning	condition
`t`	times (any one unit, e.g. s)	
`y`	transformed fractions (0 to 1)	
`y_target`	fraction whose time is wanted (optional)	

output	meaning
`n`	Avrami exponent
`k`	rate constant (time unit^-n)
`log10_k`	log10 of k
`t_half`	time to 50 % transformation (time unit of t)
`r_squared`	r^2 of the fit
`t_target`	time to the target fraction

- **Method:** least-squares line of ln(-ln(1 - y)) against ln t: slope n, intercept ln k
- **Assumes:** equal lengths, at least two distinct times; $0 < y < 1$, rising with t
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('avrami_kinetics', t=[50, 100], y=[0.2, 0.8])
# -> n = 2.85051, k = 3.20367e-06, log10_k = -5.49435, t_half = 74.4141, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Avrami kinetics: t = [50, 100], y = [0.2, 0.8]	SOLVED	n = 2.85051, k = 3.20367e-06, log10_k = -5.49435, t_half = 74.4141, r_squared = 1
Avrami kinetics: t = [50, 100], y = [0.2, 1.2]	INVALID_INPUT	Avrami (JMAK) transformation kinetics: every transformed fraction must lie between 0 and 1.

Weight percent to atomic percent (binary alloy) — `weight_to_atomic_percent`

Convert the composition of a binary alloy from weight percent to atomic percent.

$$C_1' = \frac{C_1 A_2}{C_1 A_2 + C_2 A_1} \times 100$$

input	meaning	condition
`element1`	symbol of element 1	
`element2`	symbol of element 2	
`wt1`	weight percent of element 1	

output	meaning
`at1`	atomic percent of element 1

output	meaning
`at2`	atomic percent of element 2

- **Method:** C1' = C1 A2 / (C1 $A_2 + C_2$ A1) x 100
- **Assumes:** known elements, different; $0 \leq wt1 \leq 100$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('weight_to_atomic_percent', element1='Cu', element2='Ni', wt1=60.0)
# -> at1 = 58.0793, at2 = 41.9207   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weight percent to atomic percent: element1 = Cu, element2 = Ni, wt1 = 60	SOLVED	at1 = 58.0793, at2 = 41.9207
Weight percent to atomic percent: element1 = Cu, element2 = Xx, wt1 = 60	INVALID_INPUT	Weight percent to atomic percent (binary alloy): unknown element Xx.

Atomic percent to weight percent (binary alloy) — `atomic_to_weight_percent`

Convert the composition of a binary alloy from atomic percent to weight percent.

$$C_1 = \frac{C_1' A_1}{C_1' A_1 + C_2' A_2} \times 100$$

input	meaning	condition
`element1`	symbol of element 1	
`element2`	symbol of element 2	
`at1`	atomic percent of element 1	

output	meaning
`wt1`	weight percent of element 1
`wt2`	weight percent of element 2

- **Method:** $C_1 = C_1$' A1 / (C1' $A_1 + C_2$' A2) x 100
- **Assumes:** known elements, different; $0 \leq at1 \leq 100$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('atomic_to_weight_percent', element1='Al', element2='Cu', at1=90.0)
# -> wt1 = 79.2591, wt2 = 20.7409   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Atomic percent to weight percent: element1 = Al, element2 = Cu, at1 = 90	SOLVED	wt1 = 79.2591, wt2 = 20.7409

Number- and weight-average molecular weight and dispersity — `molecular_weight_averages`

Mn, Mw and the dispersity Mw/Mn of a polymer from a molecular-weight distribution given as number fractions or weight fractions per size range.

$$\bar{M}_n = \sum x_i M_i, \quad \bar{M}_w = \sum w_i M_i, \quad \mathrm{PDI} = \bar{M}_w / \bar{M}_n$$

input	meaning	condition
`M`	mean molecular weight of each size range in g/mol	
`fraction`	number or weight fraction of each range (normalised)	
`basis`	whether the fractions are number (x_i) or weight (w_i) fractions (default number)	

output	meaning
`Mn`	number-average molecular weight in g/mol
`Mw`	weight-average molecular weight in g/mol
`PDI`	dispersity (polydispersity index) Mw/Mn

- **Method:** number basis: Mn = sum x_i M_i, Mw = sum x_i M_i^2/Mn; weight basis: Mw = sum w_i M_i, Mn = 1/sum(w_i/M_i)
- **Assumes:** equal lengths; $M > 0$; fractions >= 0, not all zero (they are normalised)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('molecular_weight_averages', M=[10000, 20000, 30000], fraction=[0.2, 0.5, 0.3], basis='number')
# -> Mn = 21000, Mw = 23333.3, PDI = 1.11111   (SOLVED)
```

```
materials_science_ai999x.solve('molecular_weight_averages', M=[10000, 20000], fraction=[0.5, 0.5], basis='weight')
# -> Mn = 13333.3, Mw = 15000, PDI = 1.125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Molecular weight averages: M = [10000, 20000, 30000], fraction = [0.2, 0.5, 0.3], basis = number	SOLVED	Mn = 21000, Mw = 23333.3, PDI = 1.11111
Molecular weight averages: M = [10000, 20000, 30000], fraction = [0.2, 0.5]	INVALID_INPUT	Number- and weight-average molecular weight and dispersity: M and fraction must have the same length.

Repeat-unit molecular weight and degree of polymerization — `degree_of_polymerization`

The molecular weight of a repeat unit from its formula and the number-average degree of polymerization $DP = Mn/m$.

$$\mathrm{DP} = \frac{\bar{M}_n}{m}$$

input	meaning	condition
`formula`	repeat-unit formula, e.g. C2H3Cl	
`Mn`	number-average molecular weight	

output	meaning
`repeat_unit_g_mol`	repeat-unit molecular weight in g/mol
`DP`	degree of polymerization

- **Method:** m = sum of atomic weights of the repeat unit; $DP = Mn/m$
- **Assumes:** a valid formula; $Mn > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('degree_of_polymerization', formula='C2H3Cl', Mn=106.0)
# -> repeat_unit_g_mol = 62.4982, DP = 1696.05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Degree of polymerization: formula = C2H3Cl, Mn = 106000 g/mol	SOLVED	repeat_unit_g_mol = 62.4982, DP = 1696.05
Degree of polymerization: formula = C2Xq3, Mn = 106000 g/mol	INVALID_INPUT	Repeat-unit molecular weight and degree of polymerization: unknown element Xq.

Average repeat-unit weight and degree of polymerization of a copolymer — `copolymer_degree_of_polymerization`

The average repeat-unit molecular weight of a copolymer from the two repeat-unit formulas and the mole fraction of the first, and its degree of polymerization.

$$\bar{m} = \sum f_j m_j, \quad \mathrm{DP} = \frac{\bar{M}_n}{\bar{m}}$$

input	meaning	condition
`formula1`	formula of repeat unit 1	
`formula2`	formula of repeat unit 2	
`f1`	mole fraction of repeat unit 1 (0 to 1)	
`Mn`	number-average molecular weight	

output	meaning
`m_avg_g_mol`	average repeat-unit molecular weight in g/mol
`DP`	degree of polymerization

- **Method:** $m = f_1\, m_1 + (1 - f_1)$ m2; $DP = Mn/m$
- **Assumes:** valid formulas; $0 \le f_1 \le 1$; $Mn > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('copolymer_degree_of_polymerization', formula1='C4H6',
formula2='C8H8', f1=0.75, Mn=350.0)
# -> m_avg_g_mol = 66.6051, DP = 5254.85   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Copolymer degree of polymerization: formula1 = C4H6, formula2 = C8H8, f1 = 0.75, Mn = 350000 g/mol	SOLVED	m_avg_g_mol = 66.6051, DP = 5254.85

Percent crystallinity of a polymer from densities — `polymer_crystallinity`

The mass percent crystallinity of a semicrystalline polymer from its density and the densities of the fully crystalline and fully amorphous polymer.

$$\%\,\mathrm{cryst} = \frac{\rho_c(\rho_s - \rho_a)}{\rho_s(\rho_c - \rho_a)} \times 100$$

input	meaning	condition
`rho_s`	density of the sample	
`rho_c`	density of the totally crystalline polymer	
`rho_a`	density of the totally amorphous polymer	

output	meaning
`crystallinity_percent`	percent crystallinity (by mass)

- **Method:** %cryst = rho_c (rho_s - rho_a) / (rho_s (rho_c - rho_a)) x 100
- **Assumes:** $rho_a < rho_s < rho_c$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('polymer_crystallinity', rho_s=925.0, rho_c=1000.0, rho_a=870.0)
# -> crystallinity_percent = 45.738   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Polymer crystallinity: rho_s = 0.925 g/cm^3, rho_c = 1.00 g/cm^3, rho_a = 0.87 g/cm^3	SOLVED	crystallinity_percent = 45.738
Polymer crystallinity: rho_s = 1.2 g/cm^3, rho_c = 1.00 g/cm^3, rho_a = 0.87 g/cm^3	INVALID_INPUT	Percent crystallinity of a polymer from densities: the sample density must lie between the amorphous and the crystalline

Stress in a stretched rubber (affine network) — `rubber_elasticity`

The shear modulus $G = \rho$ R T / Mc of a crosslinked rubber and its engineering and true stress at a uniaxial stretch ratio lambda.

$$\sigma = \frac{\rho R T}{M_c}\left(\lambda - \frac{1}{\lambda^2}\right)$$

input	meaning	condition
`rho`	density of the rubber	
`Mc`	molecular weight between crosslinks	
`T`	temperature	
`stretch`	stretch ratio $\lambda = L/L_0$	

output	meaning
`G_MPa`	shear modulus in MPa
`stress_MPa`	engineering stress in MPa
`true_stress_MPa`	true stress in MPa

- **Method:** $G = \rho$ R T / Mc; $\sigma = G\,(\lambda - 1/\lambda^2)$
- **Assumes:** all inputs > 0; ideal affine network, incompressible
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('rubber_elasticity', rho=950.0, Mc=5.0, T=300.0, stretch=2.0)
# -> G_MPa = 0.473924, stress_MPa = 0.829368, true_stress_MPa = 1.65874   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rubber elasticity: rho = 0.95 g/cm^3, Mc = 5000 g/mol, T = 300 K, stretch = 2	SOLVED	G_MPa = 0.473924, stress_MPa = 0.829368, true_stress_MPa = 1.65874
Rubber elasticity: rho = 0.95 g/cm^3, Mc = 5000 g/mol, T = 300 K, stretch = -2	INVALID_INPUT	Stress in a stretched rubber (affine network): the density, Mc, temperature and stretch must be a positive number.

End-to-end distance of a freely rotating chain — `polymer_chain_dimensions`

The root-mean-square end-to-end distance and characteristic ratio of a polymer chain of n bonds of length l with fixed bond angle and free rotation, and its all-trans contour length.

$$\langle r^2 \rangle = n l^2 \left[\frac{1+c}{1-c} - \frac{2c(1-c^n)}{n(1-c)^2}\right],\; c = -\cos\theta$$

input	meaning	condition
`n`	number of backbone bonds	
`l`	bond length	
`theta`	bond angle in degrees (default 109.47, tetrahedral)	

output	meaning
`r_rms`	rms end-to-end distance
`r_rms_nm`	rms end-to-end distance in nm

output	meaning
`C_n`	characteristic ratio <r^2>/(n l^2)
`contour_nm`	all-trans contour length in nm

- **Method:** exact finite-chain sum <r^2> = n l^2 [$(1+c)/(1-c) - 2c(1-c^n)/(n$(1 - c)^2)], $c = -\cos(\theta)$
- **Assumes:** $n \geq 1$; $l > 0$; $90 < \theta < 180$ degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('polymer_chain_dimensions', n=1, l=1.54e-10)
# -> r_rms = 1.54e-10 m, r_rms_nm = 0.154, C_n = 1, contour_nm = 0.12574   (SOLVED)
```

```
materials_science_ai999x.solve('polymer_chain_dimensions', n=2, l=1.54e-10, theta=120.0)
# -> r_rms = 2.66736e-10 m, r_rms_nm = 0.266736, C_n = 1.5, contour_nm = 0.266736   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Polymer chain dimensions: n = 2, l = 0.154 nm, theta = 120	SOLVED	r_rms = 2.66736e-10 m, r_rms_nm = 0.266736, C_n = 1.5, contour_nm = 0.266736
Polymer chain dimensions: n = 0, l = 0.154 nm	INVALID_INPUT	n: must be at least 1, got 0.

Intrinsic carrier concentration of a semiconductor — `intrinsic_carrier_concentration`

The effective densities of states Nc and Nv and the intrinsic carrier concentration $ni = sqrt$(Nc Nv) exp(-Eg/2kT) from the band gap, temperature and effective masses.

$$n_i = \sqrt{N_c N_v}\, e^{-E_g/2kT}, \quad N_c = 2\left(\frac{2\pi m_e^* kT}{h^2}\right)^{3/2}$$

input	meaning	condition
`Eg`	band gap	
`T`	temperature	
`me`	electron density-of-states effective mass / m0 (default 1)	
`mh`	hole density-of-states effective mass / m0 (default 1)	

output	meaning
`Nc`	effective density of states, conduction band
`Nv`	effective density of states, valence band
`ni`	intrinsic carrier concentration
`log10_ni`	log10 of ni in m^{-3}
`log10_Nc`	log10 of Nc in m^{-3}
`Ei_offset_meV`	intrinsic Fermi level above midgap in meV

- **Method:** $Nc = 2$(2 pi mek T/h^2)^(3/2), $ni = sqrt$(Nc Nv) exp($-Eg/(2kT)$), Ei - midgap = (3/4) k T ln(mh*/me*)
- **Assumes:** all inputs > 0; non-degenerate (Boltzmann) statistics, parabolic bands
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('intrinsic_carrier_concentration', Eg=1.7944378300800001e-19,
T=300.0, me=1.08, mh=0.81)
# -> Nc = 2.81649e+25 1/m^3, Nv = 1.82936e+25 1/m^3, ni = 8.8801e+15 1/m^3, log10_ni = 15.9484,
log10_Nc = 25.4497, Ei_offset_meV = -5.57787   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Intrinsic carrier concentration: Eg = 1.12 eV, T = 300 K, me = 1.08, mh = 0.81	SOLVED	Nc = 2.81649e+25 1/m^3, Nv = 1.82936e+25 1/m^3, ni = 8.8801e+15 1/m^3, log10_ni = 15.9484, log10_Nc = 25.4497, Ei_offset_meV = -5.57787
Intrinsic carrier concentration: Eg = 1.12 eV	NEEDS_INPUT	intrinsic_carrier_concentration needs T (temperature). Example: solve('intrinsic_carrier_concentration', Eg='1.79444e-19

Band gap from intrinsic conductivities at several temperatures — `band_gap_from_conductivity`

The band gap of an intrinsic semiconductor from its conductivity at two or more temperatures, with $\sigma = C$ exp(-Eg/2kT) (the weak temperature dependence of the prefactor neglected).

$$\ln \sigma = \ln C - \frac{E_g}{2k}\frac{1}{T}$$

input	meaning	condition
`T`	temperatures in kelvin	
`sigma`	conductivities (any one unit) at those temperatures	

output	meaning
`Eg_eV`	band gap in eV
`prefactor`	prefactor C (unit of sigma)
`r_squared`	r^2 of the fit

- **Method:** least-squares line of ln sigma against $1/T$: slope = -Eg/(2k)
- **Assumes:** equal lengths, at least two distinct temperatures; sigma rising with T
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('band_gap_from_conductivity', T=[300, 400], sigma=[2.2, 63])
# -> Eg_eV = 0.693801, prefactor = 1.47943e+06, r_squared = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Band gap from conductivity: T = [300, 400], sigma = [2.2, 63]	SOLVED	Eg_eV = 0.693801, prefactor = 1.47943e+06, r_squared = 1
Band gap from conductivity: T = [300, 400], sigma = [63, 2.2]	INVALID_INPUT	Band gap from intrinsic conductivities at several temperatures: an intrinsic conductivity rises with temperature.

Electronic thermal conductivity of a metal (Wiedemann-Franz law) — `wiedemann_franz`

The electronic thermal conductivity $\kappa = L$ sigma T of a metal from its electrical conductivity (or resistivity) and temperature, with the Sommerfeld value of the Lorenz number.

$$\kappa = L\sigma T, \quad L = \frac{\pi^2}{3}\left(\frac{k}{e}\right)^2$$

input	meaning	condition
`T`	temperature	
`sigma`	electrical conductivity (or give rho)	
`rho`	electrical resistivity (or give sigma)	

output	meaning
`kappa`	thermal conductivity
`lorenz`	Lorenz number in W ohm/K^2

- **Method:** $\kappa = L$ sigma T, $L = (pi^2/3)(k/e)^2 = 2.44e-8$ W ohm/K^2
- **Assumes:** $T > 0$; exactly one of sigma, rho, positive; degenerate free-electron metal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('wiedemann_franz', T=300.0, sigma=59600000.0)
# -> kappa = 436.809 W/(m*K), lorenz = 2.443e-08   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wiedemann-Franz: T = 300 K, sigma = 5.96e7 S/m	SOLVED	kappa = 436.809 W/(m*K), lorenz = 2.443e-08
Wiedemann-Franz: T = 300 K, sigma = 5.96e7 S/m, rho = 1.68e-8 ohm*m	INVALID_INPUT	Electronic thermal conductivity of a metal (Wiedemann-Franz law): give exactly one of the conductivity sigma and the res

Molar heat capacity of a solid (Debye model) — `debye_heat_capacity`

The constant-volume molar heat capacity of a monatomic solid from the Debye model at temperature T.

$$C_v = 9R\left(\frac{T}{\Theta_D}\right)^3 \int_0^{\Theta_D/T} \frac{x^4 e^x}{(e^x-1)^2}\,dx$$

input	meaning	condition
`theta_D`	Debye temperature	
`T`	temperature	

output	meaning
`Cv`	molar heat capacity at constant volume
`Cv_over_3R`	$Cv/3R$ (1 at high T)

- **Method:** $Cv = 9R\,(T/theta_D)^3$ int_0^(theta_D/T) $x^4\,e^x/(e^x-1)^2$ dx (mpmath quadrature, 30 digits)
- **Assumes:** theta_D, $T > 0$; $T \geq theta_D/300$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('debye_heat_capacity', theta_D=343.0, T=1000000.0)
# -> Cv = 24.9434 J/(mol*K), Cv_over_3R = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Debye heat capacity: theta_D = 343 K, T = 1000000 K	SOLVED	Cv = 24.9434 J/(mol*K), Cv_over_3R = 1
Debye heat capacity: theta_D = 343 K	NEEDS_INPUT	debye_heat_capacity needs T (temperature). Example: solve('debye_heat_capacity', theta_D='343 K', T='1e+06 K').

Molar heat capacity of a solid (Einstein model) — `einstein_heat_capacity`

The constant-volume molar heat capacity of a solid from the Einstein model (all atoms vibrating at one frequency).

$$C_v = 3R\left(\frac{\Theta_E}{T}\right)^2 \frac{e^{\Theta_E/T}}{(e^{\Theta_E/T}-1)^2}$$

input	meaning	condition
`theta_E`	Einstein temperature	
`T`	temperature	

output	meaning
`Cv`	molar heat capacity at constant volume
`Cv_over_3R`	$Cv/3R$

- **Method:** $Cv = 3R\,x^2\,e^x/(e^x-1)^2$, $x = theta_E/T$
- **Assumes:** theta_E, $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('einstein_heat_capacity', theta_E=300.0, T=300.0)
# -> Cv = 22.9647 J/(mol*K), Cv_over_3R = 0.920674   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Einstein heat capacity: theta_E = 300 K, T = 300 K	SOLVED	Cv = 22.9647 J/(mol*K), Cv_over_3R = 0.920674

Fermi energy of a free-electron metal — `fermi_energy_free_electron`

The conduction-electron density, Fermi energy, Fermi temperature and Fermi velocity of a metal from its density, molar mass and number of valence electrons per atom.

$$E_F = \frac{\hbar^2}{2m}\left(3\pi^2 n\right)^{2/3}$$

input	meaning	condition
`rho`	density	
`A`	molar mass	
`valence`	valence electrons per atom	

output	meaning
n	conduction-electron density
`log10_n`	log10 of n in m^{-3}
`EF`	Fermi energy
`EF_eV`	Fermi energy in eV
`TF`	Fermi temperature
`vF`	Fermi velocity

- **Method:** $n = z$ N_A rho/A; E_F = (hbar^2/2m)(3 pi^2 n)^(2/3)
- **Assumes:** all inputs > 0; free-electron model
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('fermi_energy_free_electron', rho=8960.0, A=0.06355, valence=1)
# -> n = 8.4907e+28 1/m^3, log10_n = 28.9289, EF = 1.12861e-18 J, EF_eV = 7.04421, TF = 81744.7 K,
vF = 1.57413e+06 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fermi energy: rho = 8.96 g/cm^3, A = 63.55 g/mol, valence = 1	SOLVED	n = 8.4907e+28 1/m^3, log10_n = 28.9289, EF = 1.12861e-18 J, EF_eV = 7.04421, TF = 81744.7 K, vF = 1.57413e+06 m/s
Fermi energy: rho = 8.96 g/cm^3, A = 63.55 g/mol	NEEDS_INPUT	fermi_energy_free_electron needs valence (valence electrons per atom). Example: solve('fermi_energy_free_electron', rho=

Diffusion coefficient at a temperature (Arrhenius) — `arrhenius_diffusivity`

The diffusion coefficient $D = D_0$ exp(-Q_d/RT) at temperature T from the pre-exponential factor and the activation energy.

$$D = D_0 \exp\left(-\frac{Q_d}{RT}\right)$$

input	meaning	condition
`D0`	pre-exponential factor	
`Q`	activation energy per mole	
`T`	absolute temperature	

output	meaning
`D`	diffusion coefficient
`log10_D`	log10 of D in m^2/s
`Q_over_RT`	Q_d/(R T)

- **Method:** $D = D_0$ exp(-Q_d/(R T)), $R = N_A$ k
- **Assumes:** D0, Q, $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('arrhenius_diffusivity', D0=2.3e-05, Q=148000.0, T=773.0)
# -> D = 2.29606e-15 m^2/s, log10_D = -14.639, Q_over_RT = 23.0276   (SOLVED)
```

```
materials_science_ai999x.solve('arrhenius_diffusivity', D0=0.0001, Q=19144.757680815896, T=1000.0)
# -> D = 1e-05 m^2/s, log10_D = -5, Q_over_RT = 2.30259   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Arrhenius diffusivity: D0 = 2.3e-5 m^2/s, Q = 148 kJ/mol, T = 773 K	SOLVED	D = 2.29606e-15 m^2/s, log10_D = -14.639, Q_over_RT = 23.0276
Arrhenius diffusivity: D0 = 2.3e-5 m^2/s, Q = 148 kJ/mol, T = -10 K	INVALID_INPUT	Diffusion coefficient at a temperature (Arrhenius): D0, Q and T must be a positive number.
Arrhenius diffusivity: D0 = 2.3e-5 m^2/s, Q = 148 kJ/mol	NEEDS_INPUT	arrhenius_diffusivity needs T (absolute temperature). Example: solve('arrhenius_diffusivity', D0='2.3e-05 m^2/s', Q='148

Larson-Miller parameter (creep rupture) — `larson_miller_parameter`

The Larson-Miller parameter LMP = T (C + log10 t_r) from a rupture time at a temperature, and the rupture time at another temperature for the same parameter (same stress).

$$\mathrm{LMP} = T\,(C + \log_{10} t_r)$$

input	meaning	condition
`T`	absolute temperature	
`t_r`	rupture time (hours, h ...)	
`C`	Larson-Miller constant (default 20, with t_r in hours)	
`T2`	another temperature at the same stress (optional)	

output	meaning
`LMP`	Larson-Miller parameter (K, t in hours)
`LMP_thousands`	LMP / 1000
`log10_t_hours`	log10 of the rupture time in hours
`t_r2_hours`	rupture time at T2 in hours
`log10_t2_hours`	log10 of the rupture time at T2

- **Method:** LMP = T (C + log10 t_r[h]); at T2: log10 t_2 = LMP/T2 - C
- **Assumes:** T, $t_r > 0$; $5 \le C \le 50$; time in hours inside the logarithm
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('larson_miller_parameter', T=1073.0, t_r=3600000.0)
# -> LMP = 24679, LMP_thousands = 24.679, log10_t_hours = 3   (SOLVED)
```

```
materials_science_ai999x.solve('larson_miller_parameter', T=1000.0, t_r=360000.0, C=20.0,
T2=1100.0)
# -> LMP = 22000, LMP_thousands = 22, log10_t_hours = 2, t_r2_hours = 1, log10_t2_hours = 0
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Larson-Miller parameter: T = 1073 K, t_r = 1000 h	SOLVED	LMP = 24679, LMP_thousands = 24.679, log10_t_hours = 3
Larson-Miller parameter: T = 1000 K, t_r = 100 h, C = 20, T2 = 1100 K	SOLVED	LMP = 22000, LMP_thousands = 22, log10_t_hours = 2, t_r2_hours = 1, log10_t2_hours = 0
Larson-Miller parameter: T = 1000 K, t_r = 100 h, C = 80	INVALID_INPUT	Larson-Miller parameter (creep rupture): the Larson-Miller constant C must lie between 5 and 50 (typically 20).
Larson-Miller parameter: T = 1073 K	NEEDS_INPUT	larson_miller_parameter needs t_r (rupture time (hours, h ...)). Example: solve('larson_miller_parameter', T='1073 K', t

Linear thermal expansion — `linear_thermal_expansion`

The length change $dL = \alpha$ L0 dT and the new length of a bar heated or cooled freely by dT.

$$\Delta L = \alpha L_0 \, \Delta T$$

input	meaning	condition
`alpha`	linear thermal expansion coefficient	

input	meaning	condition
`L0`	initial length	
`dT`	temperature change (positive = heating)	

output	meaning
`dL`	length change
`dL_mm`	length change in mm
`L`	new length
`strain`	thermal strain alpha dT

- **Method:** $dL = \alpha$ L0 dT, $L = L_0 + dL$
- **Assumes:** alpha, $L_0 > 0$; alpha constant over the range (linear, small strain)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('linear_thermal_expansion', alpha=1.7e-05, L0=0.5, dT=100.0)
# -> dL = 0.00085 m, dL_mm = 0.85, L = 0.50085 m, strain = 0.0017   (SOLVED)
```

```
materials_science_ai999x.solve('linear_thermal_expansion', alpha=2.36e-05, L0=10.0, dT=-50.0)
# -> dL = -0.0118 m, dL_mm = -11.8, L = 9.9882 m, strain = -0.00118   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Linear thermal expansion: alpha = 17e-6 1/K, L0 = 0.5 m, dT = 100 K	SOLVED	dL = 0.00085 m, dL_mm = 0.85, L = 0.50085 m, strain = 0.0017
Linear thermal expansion: alpha = 17e-6 1/K, L0 = -0.5 m, dT = 100 K	INVALID_INPUT	Linear thermal expansion: alpha and L0 must be a positive number.
Linear thermal expansion: alpha = 17e-6 1/K, L0 = 0.5 m	NEEDS_INPUT	linear_thermal_expansion needs dT (temperature change (positive = heating)). Example: solve('linear_thermal_expansion',

Volumetric thermal expansion — `volumetric_thermal_expansion`

The volume change $dV = \beta$ V0 dT of an isotropic solid with $\beta = 3$ alpha (linearised), and the exact change V0((1 + alpha dT)^3 - 1).

$$\Delta V = \beta V_0 \, \Delta T, \quad \beta = 3\alpha$$

input	meaning	condition
`alpha`	LINEAR thermal expansion coefficient	
`V0`	initial volume	
`dT`	temperature change (positive = heating)	

output	meaning
`beta`	volumetric coefficient 3 alpha
`dV`	linearised volume change beta V0 dT

output	meaning
`dV_cm3`	linearised volume change in cm^3
`V`	new volume (linearised)
`dV_exact`	exact volume change V0((1 + alpha dT)^3 - 1)
`dV_exact_cm3`	exact volume change in cm^3

- **Method:** $\beta = 3$ alpha, $dV = \beta$ V0 dT; exact: V0((1 + alpha dT)^3 - 1)
- **Assumes:** alpha, $V_0 > 0$; isotropic solid, alpha constant over the range
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('volumetric_thermal_expansion', alpha=1.7e-05, V0=0.001, dT=100.0)
# -> beta = 5.1e-05 1/K, dV = 5.1e-06 m^3, dV_cm3 = 5.1, V = 0.0010051 m^3, dV_exact = 5.10867e-06
m^3, dV_exact_cm3 = 5.10867   (SOLVED)
```

```
materials_science_ai999x.solve('volumetric_thermal_expansion', alpha=1e-05, V0=1.0, dT=-10.0)
# -> beta = 3e-05 1/K, dV = -0.0003 m^3, dV_cm3 = -300, V = 0.9997 m^3, dV_exact = -0.00029997 m^3,
dV_exact_cm3 = -299.97   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Volumetric thermal expansion: alpha = 17e-6 1/K, V0 = 1000 cm^3, dT = 100 K	SOLVED	beta = 5.1e-05 1/K, dV = 5.1e-06 m^3, dV_cm3 = 5.1, V = 0.0010051 m^3, dV_exact = 5.10867e-06 m^3, dV_exact_cm3 = 5.10867
Volumetric thermal expansion: alpha = 17e-6 1/K, V0 = 0 cm^3, dT = 100 K	INVALID_INPUT	Volumetric thermal expansion: alpha and V0 must be a positive number.
Volumetric thermal expansion: alpha = 17e-6 1/K, dT = 100 K	NEEDS_INPUT	volumetric_thermal_expansion needs V0 (initial volume). Example: solve('volumetric_thermal_expansion', alpha='1.7e-05 1/

Electron and hole concentrations of a doped semiconductor —

`extrinsic_carrier_concentration`

The electron and hole concentrations of a semiconductor with donor and acceptor concentrations Nd and Na (complete ionisation), from charge neutrality and n $p = ni^2$, and the Fermi level relative to the intrinsic level.

$$n = \frac{N_d - N_a}{2} + \sqrt{\left(\frac{N_d - N_a}{2}\right)^2 + n_i^2}, \quad p = \frac{n_i^2}{n}$$

input	meaning	condition
`Nd`	donor concentration	
`Na`	acceptor concentration	
`ni`	intrinsic carrier concentration	
`T`	temperature (optional, for the Fermi level)	

output	meaning
`n`	electron concentration
`p`	hole concentration
`log10_n`	log10 of n in m^{-3}
`log10_p`	log10 of p in m^{-3}
`carrier_type`	n-type, p-type or intrinsic
`EF_minus_Ei_eV`	Fermi level above the intrinsic level in eV (when T is given)

- **Method:** $n - p = Nd - Na$, n $p = ni^2$: majority = |Nd - Na|/$2 + sqrt$((Nd - Na)^2/4 + ni^2), minority = ni^2/majority; $E_F - E_i = kT$ ln(n/ni)
- **Assumes:** Nd, $Na \geq 0$; $ni > 0$; complete ionisation, non-degenerate statistics
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('extrinsic_carrier_concentration', Nd=1e+22, Na=2e+21, ni=1e+16)
# -> n = 8e+21 1/m^3, p = 1.25e+10 1/m^3, log10_n = 21.9031, log10_p = 10.0969, carrier_type = n-
type   (SOLVED)
```

```
materials_science_ai999x.solve('extrinsic_carrier_concentration', Nd=0.0, Na=3e+22, ni=4e+22)
# -> n = 2.772e+22 1/m^3, p = 5.772e+22 1/m^3, log10_n = 22.4428, log10_p = 22.7613, carrier_type =
p-type   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Extrinsic carrier concentration: Nd = 1e22 m^-3, Na = 2e21 m^-3, ni = 1e16 m^-3	SOLVED	n = 8e+21 1/m^3, p = 1.25e+10 1/m^3, log10_n = 21.9031, log10_p = 10.0969, carrier_type = n-type
Extrinsic carrier concentration: Nd = 1e22 m^-3, Na = 2e21 m^-3, ni = 0 m^-3	INVALID_INPUT	Electron and hole concentrations of a doped semiconductor: the intrinsic concentration must be a positive number.
Extrinsic carrier concentration: Nd = 1e22 m^-3, Na = 2e21 m^-3	NEEDS_INPUT	extrinsic_carrier_concentration needs ni (intrinsic carrier concentration). Example: solve('extrinsic_carrier_concentrat

Built-in potential of a p-n junction — `pn_junction_built_in_potential`

The built-in potential $V_bi = (kT/q)$ ln(n_n p_p/ni^2) of an abrupt p-n junction (majority concentrations from charge neutrality; V_bi ~ (kT/q) ln(Na Nd/ni^2) when Na, Nd >> ni), and with a permittivity the zero-bias depletion width.

$$V_{bi} = \frac{kT}{q} \ln \frac{N_a N_d}{n_i^2}, \quad W = \sqrt{\frac{2\varepsilon V_{bi}}{q}\left(\frac{1}{N_a} + \frac{1}{N_d}\right)}$$

input	meaning	condition
`Na`	acceptor concentration on the p side	
`Nd`	donor concentration on the n side	

input	meaning	condition
`ni`	intrinsic carrier concentration	
`T`	temperature	
`eps_r`	relative permittivity of the semiconductor (optional, for the depletion width)	

output	meaning
`V_bi`	built-in potential
`V_bi_approx`	(kT/q) ln(Na Nd/ni^2)
`thermal_voltage`	kT/q
`W`	zero-bias depletion width
`W_um`	depletion width in um
`x_n_um`	depletion width on the n side in um
`x_p_um`	depletion width on the p side in um

- **Method:** $V_bi = (kT/q)$ ln(n_n p_p/ni^2); $W = sqrt$(2 eps V_bi (1/Na + 1/Nd)/q), $x_n = W$ Na/(Na + Nd)
- **Assumes:** Na, Nd, ni, $T > 0$; abrupt junction, complete ionisation, non-degenerate; depletion approximation for W
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
materials_science_ai999x.solve('pn_junction_built_in_potential', Na=1e+23, Nd=1e+22, ni=1e+16,
T=300.0)
# -> V_bi = 0.773844 V, V_bi_approx = 0.773844 V, thermal_voltage = 0.025852 V   (SOLVED)
```

```
materials_science_ai999x.solve('pn_junction_built_in_potential', Na=1e+23, Nd=1e+23, ni=1e+16,
T=300.0, eps_r=11.7)
# -> V_bi = 0.83337 V, V_bi_approx = 0.83337 V, thermal_voltage = 0.025852 V, W = 1.46812e-07 m,
W_um = 0.146812, x_n_um = 0.0734059, x_p_um = 0.0734059   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Built-in potential: Na = 1e23 m^-3, Nd = 1e22 m^-3, ni = 1e16 m^-3, T = 300 K	SOLVED	V_bi = 0.773844 V, V_bi_approx = 0.773844 V, thermal_voltage = 0.025852 V
Built-in potential: Na = 1e23 m^-3, Nd = 1e22 m^-3, ni = 1e16 m^-3, T = 0 K	INVALID_INPUT	Built-in potential of a p-n junction: Na, Nd, ni and T must be a positive number.
Built-in potential: Na = 1e23 m^-3, Nd = 1e22 m^-3, T = 300 K	NEEDS_INPUT	pn_junction_built_in_potential needs ni (intrinsic carrier concentration). Example: solve('pn_junction_built_in_potentia